

Chapter

04

지금까지 클로드 웹 인터페이스에서 프롬프트를 작성하고, 클로드 코드를 설치해 터미널에서 프로그램을 만들어 보았습니다. 이제 이 두 가지 환경을 함께 활용하여 보다 체계적인 개발 프로세스를 경험해 보겠습니다. 클로드 웹에서 체계적으로 프롬프트를 설계하고, 이를 클로드 코드에서 실행하는 효율적인 워크플로를 익혀 보겠습니다. 특히 클로드 코드의 다양한 명령어들을 실제 프로젝트를 만들면서 자연스럽게 습득하게 될 것입니다.

클로드 코드 실전 활용

학습목표

- 클로드 웹과 클로드 코드를 연계하여 효율적으로 작업할 수 있습니다.
- 클로드 코드의 기본 명령어를 상황에 맞게 활용할 수 있습니다.
- 체계적인 프롬프트 설계로 개발 효율성을 높일 수 있습니다.
- 실용적인 할 일 관리 앱을 완성할 수 있습니다.

04-1

단계별 프롬프트와 함께 클로드 코드 명령어 익히기

핵심 키워드

단계별 프롬프트 전략

/init

/status

/model

/help

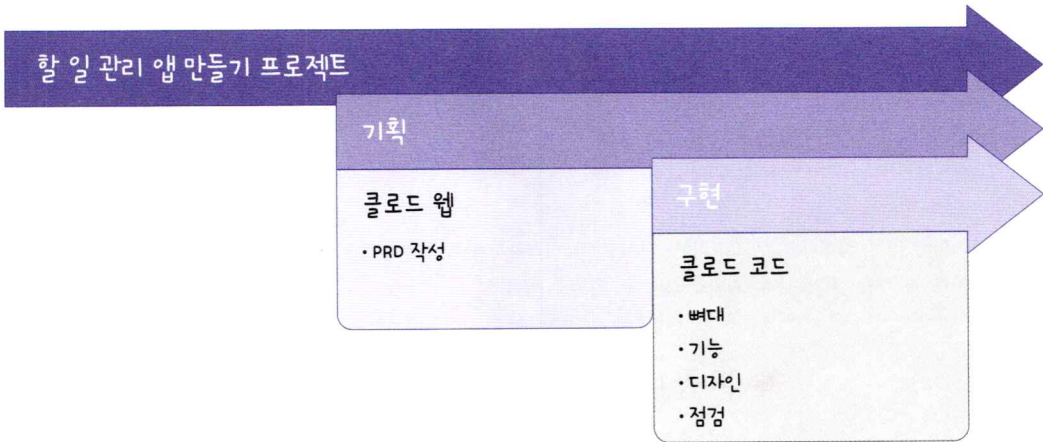
이번 절의 목표는 기획 - 구현의 워크플로를 자연스럽게 체득하는 것입니다. 실제 프로젝트에 PRD 작성법과 5W1H 프레임워크를 적용해 '할 일 관리 앱'의 요구사항을 정리하고, 단계별 프롬프트로 변환합니다. 이를 클로드 코드에서 구현하며 기본 명령어 사용법도 함께 익힐 것입니다.

시작하기 전에

이번 절에서 만들 앱은 개인용 할 일 관리 애플리케이션입니다. 매일 해야 할 일을 간단히 기록하고 완료 여부를 체크할 수 있으며, 업무/개인/공부와 같이 카테고리별로 항목을 분류해 관리할 수 있게 만들 것입니다. 앱은 웹 브라우저에서 바로 실행되며, 복잡한 프레임워크 대신 순수 HTML, CSS, 자바스크립트만으로 진행해 초보자도 바로 따라 할 수 있습니다.



이번 장에서는 할 일 관리 앱을 만들면서 클로드 코드의 여러 명령어를 실습해 볼 예정입니다. 프로젝트의 워크플로를 기획과 구현 두 단계로 나누고, 각각 클로드와 클로드 코드에서 진행하겠습니다.



클로드가 생성하는 PRD와 단계별 프롬프트는 매번 조금씩 다르게 나타날 수 있습니다. 책에 실린 예시와 동일하지 않아도 괜찮습니다. 중요한 것은 PRD라는 형식을 지켜 체계적으로 요구사항을 정리하고, 이를 단계별로 실행해 보는 경험입니다. 각자의 결과물을 그대로 활용해도 실습에는 전혀 문제가 없습니다.

단계별 프롬프트 작성하기

2장에서 익힌 5W1H 프레임워크를 활용해 체계적으로 프로젝트 요구사항을 정리하고, PRD로 정리해 보겠습니다. 이 PRD는 앞으로 만들 할 일 관리 앱의 설계도와 같으며, 개발 방향을 명확히 잡아줍니다. PRD를 기반으로 **단계별 프롬프트 전략**을 도출하여 이후 클로드 코드에서 실제 구현에 활용해 보겠습니다.

PRD 작성하기

01 웹 브라우저에서 클로드(claude.ai)에 접속하여 새로운 대화를 시작하고 할 일 관리 앱의 PRD를 작성하기 위해 다음과 같이 프롬프트를 입력합니다.

프롬프트

할 일 관리 앱을 만들고 싶어. PRD를 작성해줘.

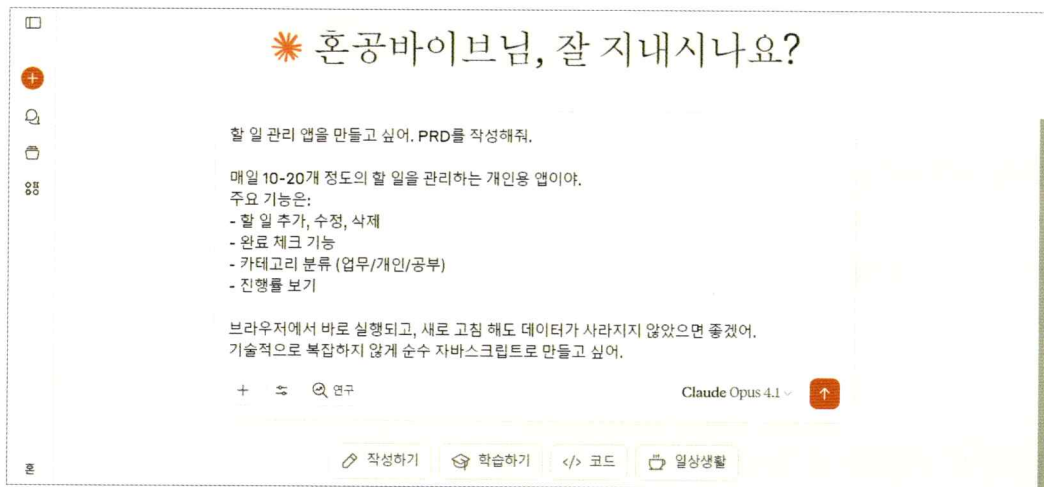
매일 10-20개 정도의 할 일을 관리하는 개인용 앱이야.

주요 기능은:

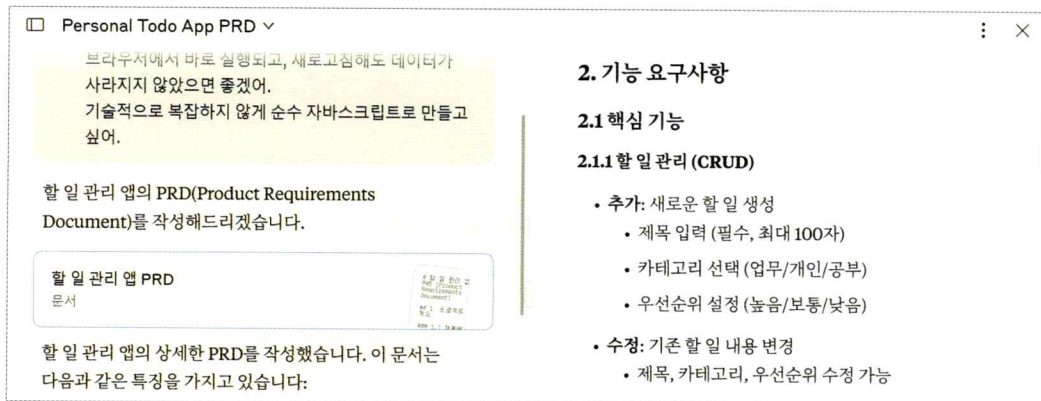
- 할 일 추가, 수정, 삭제
- 완료 체크 기능
- 카테고리 분류 (업무/개인/공부)
- 진행률 보기

브라우저에서 바로 실행되고, 새로고침해도 데이터가 사라지지 않았으면 좋겠어.

기술적으로 복잡하지 않게 순수 자바스크립트로 만들고 싶어.



02 클로드가 체계적인 PRD 작성을 시작합니다. PRD의 구조와 핵심 요구사항을 확인하세요.



note 생성형 AI의 특성상 매번 다른 표현과 구조로 응답할 수 있지만, PRD라는 포맷에 맞추어 체계적으로 정리되므로 핵심 내용은 유사하게 나올 것입니다.

프롬프트로 변환하기

이제 이 PRD를 클로드 코드에서 실행할 수 있는 구체적인 프롬프트로 변환해 봅시다. 클로드 코드의 프롬프트는 웹과 달리 파일 생성과 수정이 가능하다는 것에 초점을 맞추면서 더 구체적이고 실행 가능한 형태로 작성합니다.

생성한 PRD를 바탕으로 단계별 프롬프트 전략을 구축해 봅시다. 클로드에 다음과 같이 요청하세요. 클로드가 명확한 지시사항과 구체적인 파일 작업 명령을 정리해 줍니다.

프롬프트

이 PRD를 클로드 코드에서 사용할 단계별 프롬프트로 변환해 줘.
5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘.

□ Personal Todo App PRD ▾

H 이 PRD를 클로드 코드에서 사용할 단계별 프롬프트로 변환해 줘.
5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘.

PRD를 바탕으로 Claude Code에서 사용할 수 있는 5단계 개발 프롬프트를 만들어드리겠습니다.

할 일 관리 앱 개발 단계별 프롬프트 문서

PRD를 바탕으로 Claude Code에서 사용할 수 있는 5단계 개발 프롬프트를 만들었습니다 ↓

Claude에게 답변하기

Sonnet 4 ▾

↑

할 일 관리 앱 개발 - Claude Code 단계별 프롬프트

🚩 Step 1: 프로젝트 구조 및 기본 HTML 구성

할 일 관리 앱의 기본 구조를 만들어주세요.

요구사항:

1. ****프로젝트 구조**:**

- index.html (메인 페이지)
- styles.css (스타일시트)
- script.js (JavaScript 로직)
- README.md (프로젝트 설명)

2. ****HTML 구성**:**

- 시멘틱 HTML5 구조 사용
- 메타 태그 및 반응형 viewport 설정
- 다음 섹션들을 포함:

note 클로드 웹에서 만든 단계별 프롬프트를 메모장이나 다른 텍스트 편집기에 저장해두고, 클로드 코드에서 작업할 때 복사해서 사용합니다. 웹에서는 '기획과 설계'를, 코드에서는 '구현'을 담당하는 효율적인 워크플로를 만들 수 있습니다.

이렇게 단계별로 정리된 프롬프트를 받으면, 이제 클로드 코드에서 하나씩 실행할 준비가 완료된 것입니다. 각 단계는 독립적으로 실행 가능하며, 이전 단계의 결과물 위에 점진적으로 기능을 추가하는 방식으로 구성됩니다.

클로드로 생성한 PRD와 단계별 프롬프트는 사용자마다 상이합니다. 이 책과 내용이 달라도 괜찮습니다.

+ 여기서 잠깐

이번 장에서 참고할 것

6장에서는 클로드 코드에서 직접 PRD를 작성하고 단계별 프롬프트를 생성해 md 파일로 저장해 활용하는 방법을 배웁니다. 이번 절에서는 본격적인 실습에 앞서, 익숙한 웹 인터페이스 환경에서 생성된 프롬프트를 확인하며 기본 개념과 작업 흐름을 익히는 데 중점을 둡니다.



단계별 프롬프트 실행하기

앞서 클로드 웹에서 준비한 프롬프트를 단계별로 실행해 봅시다. 텍스트를 복사해서 클로드 코드에 그대로 붙여 넣으면 됩니다.

Personal Todo App PRD ▾

5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을 만들어 줘.

PRD를 바탕으로 Claude Code에서 사용할 수 있는 5단계 개발 프롬프트를 만들어드리겠습니다.

할 일 관리 앱 개발 단계별 프롬프트 문서

PRD를 바탕으로 Claude Code에서 사용할 수 있는 5단계 개발 프롬프트를 만들었습니다!

각 단계의 특징:

1. Step 1 (프로젝트 구조): 기본 파일 구조와 HTML 마크업 생성

2. Step 2 (스타일링): 전체 디자인 시스템과 반응형 CSS 구현

3. Step 3 (데이터 관리): 핵심 비즈니스 로직과 localStorage 처리

Claude에게 답변하기

Claude Sonnet 4 ▾

↑

복사 ▾

게시 ▾

×

할 일 관리 앱 개발 - Claude Code 단계별 프롬프트

Step 1: 프로젝트 구조 및 기본 HTML 구성

할 일 관리 앱의 기본 구조를 만들어주세요.

요구사항:

- 프로젝트 구조**:
 - index.html (메인 페이지)
 - styles.css (스타일시트)
 - script.js (JavaScript 로직)
 - README.md (프로젝트 설명)
- HTML 구성**:
 - 시멘틱 HTML5 구조 사용
 - 메타 태그 및 반응형 viewport 설정
 - 다음 섹션들을 포함:
 - * 헤더: 앱 제목 "PersonalTodo"
 - * 진행률 표시 영역 (전체 및 카테고리별)
 - * 할 일 추가 폼 (제목, 카테고리, 우선순위)
 - * 필터 버튼들 (전체, 업무, 개인, 공부)
 - * 할 일 목록 컨테이너

note 클로드 아티팩트의 '복사' 아이콘을 클릭해도 됩니다.

+ 여기서 잠깐

프롬프트 자료실

이 절부터는 사용자가 직접 입력하는 프롬프트와 교재에서 제시하는 예시 프롬프트가 서로 다를 수 있습니다. 학습에 참고하고 싶다면 이 책의 깃허브 자료실(<https://github.com/taehojo/vibecoding>)에서 [전체 프롬프트 모음 보기]를 찾아 클릭하세요. 이 책에서 사용된 프롬프트가 장별로 정리되어 제공됩니다.

README

✎

☰

Deployments 276

혼자 공부하는 바이브코딩 with 클로드코드

실습 결과 일부 미리보기

이미지를 클릭하면 라이브 데모를 바로 실행할 수 있습니다

3장 예제: 손글씨 인식

6장 예제: 냉장고를 부탁해

7장 예제: 공감 AI 다이어리

전체 프롬프트 모음 보기

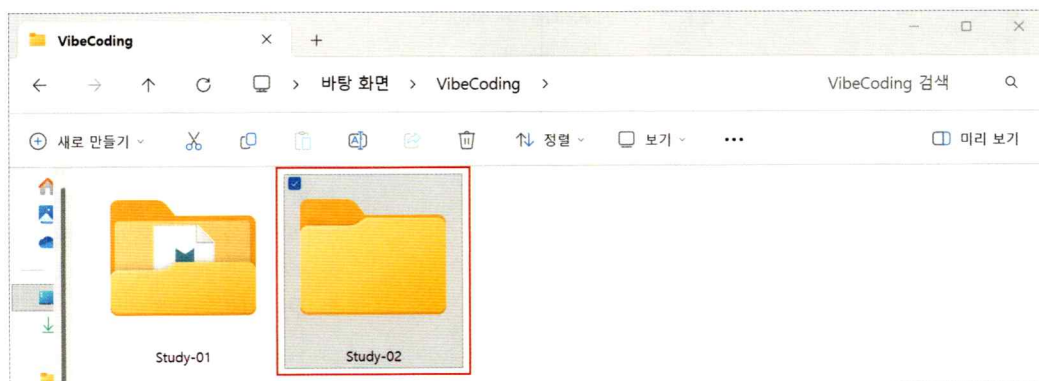
교재의 안내에 따라 프롬프트를 사용하세요.

깃허브 자료실 바로가기

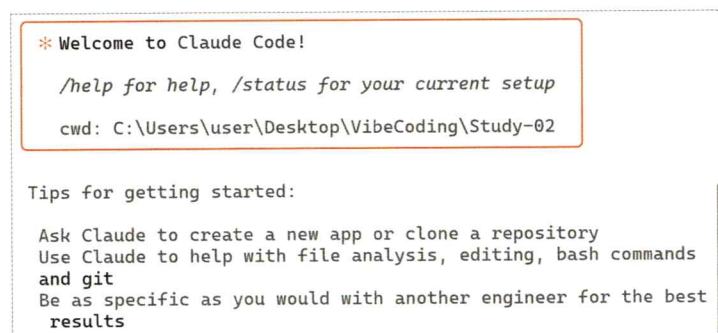
QR 코드

클로드 코드 실행하고 초기 설정하기

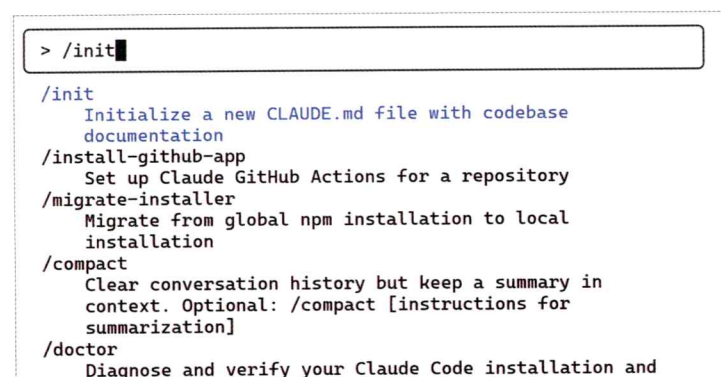
01 데스크톱의 'VibeCoding' 폴더에 새로운 폴더 'Study-02'를 생성한 뒤, 더블 클릭해 폴더를 엽니다.



02 'Study-02' 폴더의 빈 공간에 마우스 오른쪽 버튼을 누른 뒤 메뉴에서 [터미널에서 실행]을 클릭해 명령 프롬프트를 실행합니다. 그리고 나서 'claude'를 입력해서 클로드 코드를 실행합니다.

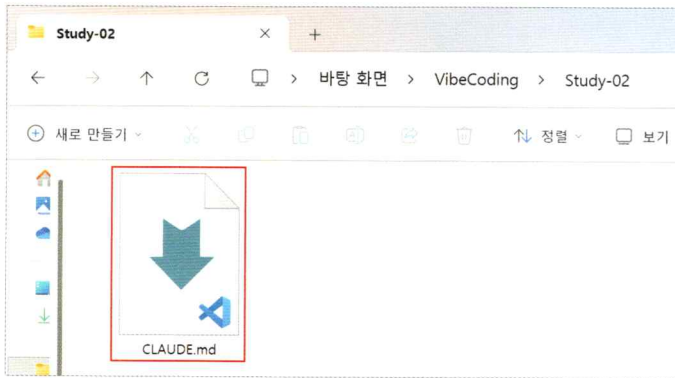


03 클로드 코드가 실행되면 '/init' 명령어를 입력하여 프로젝트 초기 설정을 시작합니다.



note /init 실행 중 Git 저장소 확인, 폴더 읽기, 파일 생성 등 여러 권한 요청 메시지가 나타날 수 있습니다. '1. Yes'를 선택해 계속 진행하세요.

04 현재 폴더를 분석하여 CLAUDE.md 파일을 자동으로 생성합니다. 아직 아무 파일도 없는 빈 폴더 상태이므로 CLAUDE.md 파일에는 기본적인 프로젝트 설정만 담기게 될 것입니다.



note /init이 상위 폴더를 분석하여 관련 없는 내용이 포함되어도 걱정할 필요 없습니다. 프로젝트가 진행되면서 실제 할 일 관리 앱 관련 내용이 CLAUDE.md 파일에 추가되고, 초기의 잘못된 추측은 자연스럽게 무시되거나 새 내용으로 대체됩니다.

05 앞서 클로드 웹에서 작성한 단계별 프롬프트를 준비하세요. 클로드가 제시한 다섯 단계를 따라 할 일 관리 앱을 만들어 볼 것입니다.

- **Step 1:** 기본 구조 및 핵심 기능 구현
- **Step 2:** 카테고리 기능 및 UI 개선
- **Step 3:** 진행률 대시보드 추가
- **Step 4:** 다크 모드 및 고급 기능
- **Step 5:** 최종 완성 및 최적화

여러분의 실습 과정이 이 책과 달라도 괜찮습니다. 클로드가 제시한 단계별 프롬프트를 따라 진행하세요.



1단계: 기본 구조 및 핵심 기능 구현

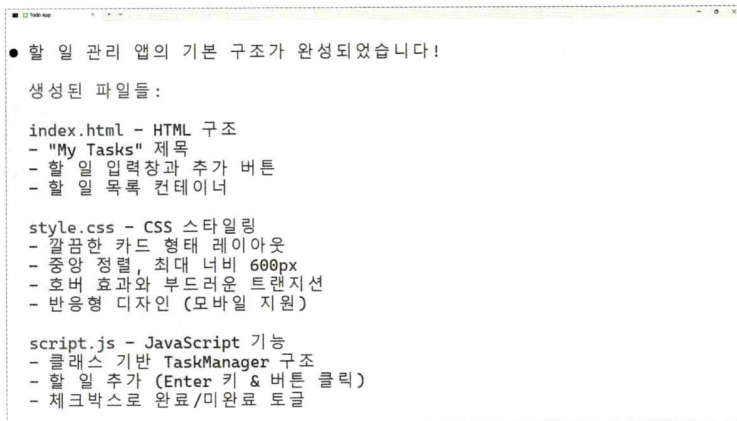
01 단계별 프롬프트의 첫 번째 프롬프트(Step 1)를 복사해 클로드 코드가 실행 중인 터미널에 **Ctrl** + **V** 키를 눌러 붙여 넣습니다.

```
> Step 1: 기본 구조 및 핵심 기능 구현
할 일 관리 앱의 기본 구조를 만들어줘.

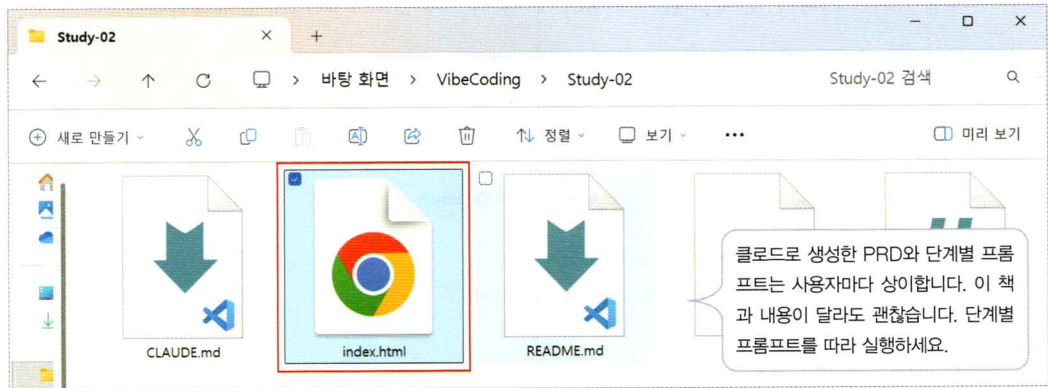
요구사항:
1. index.html, style.css, script.js 세 개의 파일로 구성
2. HTML 구조:
  - 앱 제목 "My Tasks"
  - 할 일 입력창 (input + 추가 버튼)
  - 할 일 목록을 표시할 컨테이너
3. JavaScript 기능:
  - 할 일 추가 (Enter 키와 버튼 클릭 모두 지원)
  - 할 일 삭제 (각 항목에 X 버튼)
  - 체크박스로 완료/미완료 토글
  - 완료 시 취소선 스타일 적용
4. localStorage에 데이터 저장:
  - 추가/삭제/완료 상태 변경 시 자동 저장
  - 페이지 새로고침 시 데이터 유지
5. 기본 CSS 스타일링:
  - 깔끔한 카드 형태의 레이아웃
```

note 터미널에서 여러 행의 프롬프트를 직접 입력할 때는 **\n** + **Enter** 또는 **Ctrl** + **J** 키를 눌러 줄바꿈할 수 있습니다.

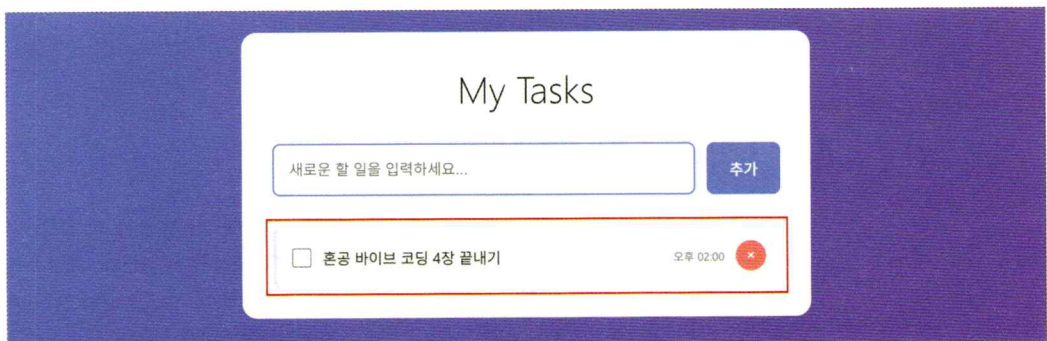
02 진행되는 내용을 확인하며 기다리면 곧 기본 구조가 구현되었다는 메시지를 만날 수 있습니다.



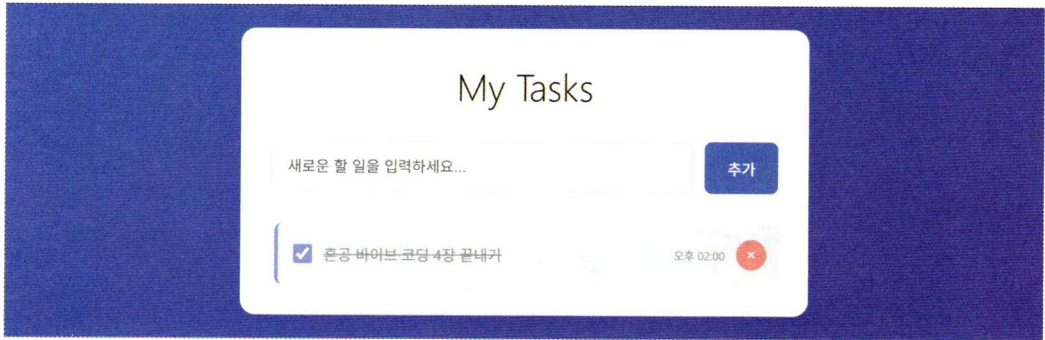
03 데스크톱의 파일 탐색기에 생성된 'index.html' 파일을 클릭해 실행합니다.



04 기본 브라우저가 열리며, 할 일 관리 앱이 열립니다. 기능을 테스트하기 위해 [새로운 할 일을 입력하세요...] 입력창에 '혼공 바이브 코딩 4장 끝내기'라고 입력한 뒤 [추가] 버튼을 눌러 할 일을 추가해 봅시다. 할일이 잘 추가되는 것을 확인합니다.

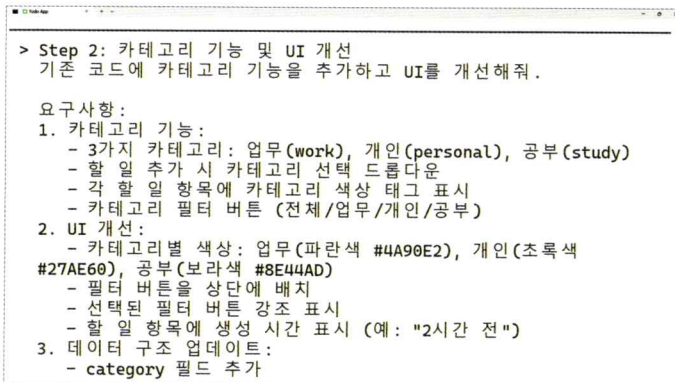


05 체크 박스를 클릭해 완료된 할 일로 전환하거나, [X] 버튼을 클릭해 삭제하는 등 다른 기능도 테스트합니다.

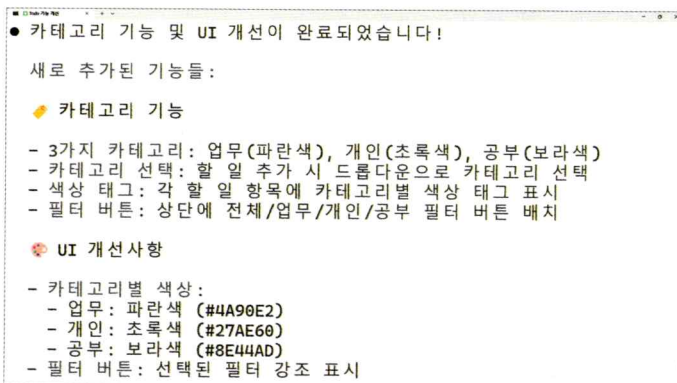


2단계: 카테고리 기능 및 UI 개선

01 단계별 프롬프트의 두 번째 프롬프트(Step 2)를 복사해 클로드 코드에 입력합니다.

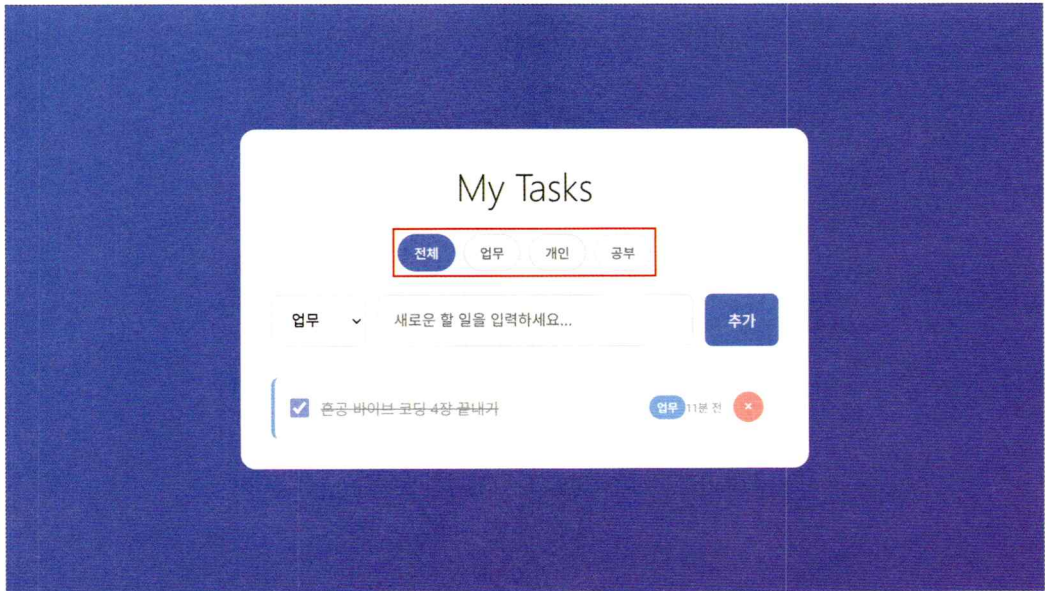


02 완료되었다는 메시지가 나타나면 진행 상황을 확인합니다.

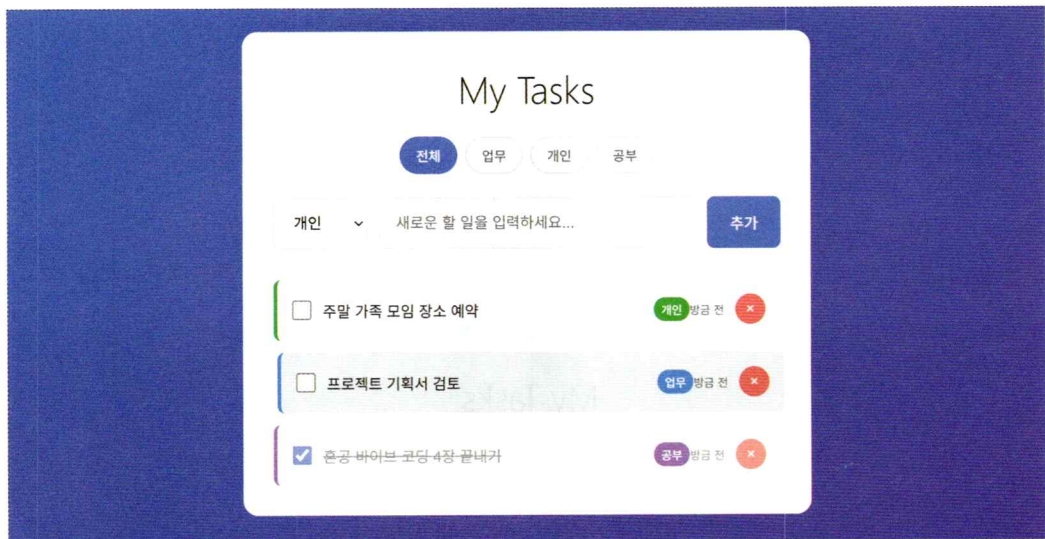


note 중간에 권한을 묻는 메시지가 나타나면 '1. Yes'를 선택하세요. 매번 누르기 번거롭다면 '2. Yes, and don't ask again ...'을 선택해도 좋습니다.

03 'index.html' 파일을 새로 열거나 이미 열려 있는 브라우저를 새로고침하세요. 업무, 개인, 공부 카테고리가 생겼고, 전체 카테고리를 한눈에 볼 수 있게 UI가 개선되었습니다.

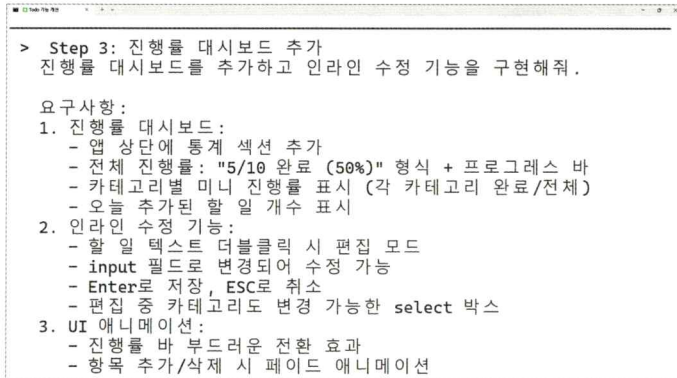


04 카테고리별로 입력해 보면 잘 분류되어 기록되는 것을 확인할 수 있습니다.

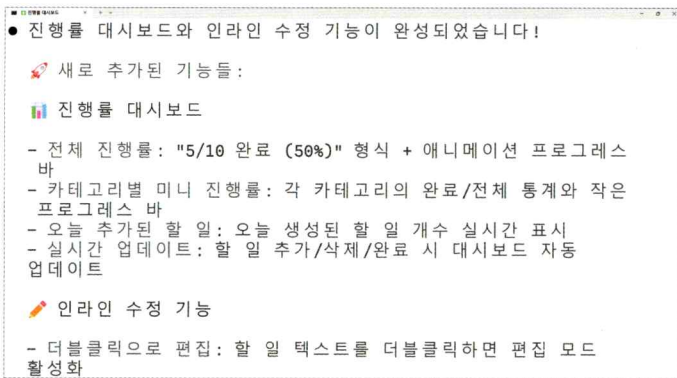


3단계: 진행률 대시보드 추가

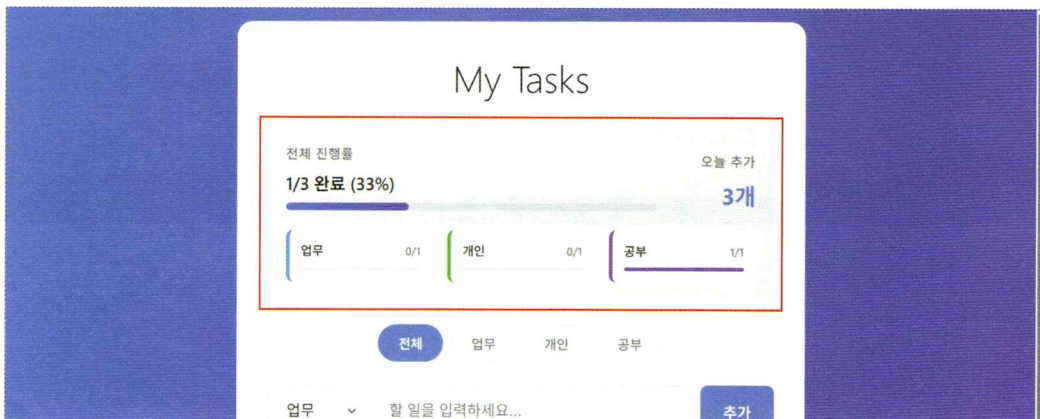
01 단계별 프롬프트의 세 번째 프롬프트(Step 3)를 복사해 클로드 코드에 입력합니다



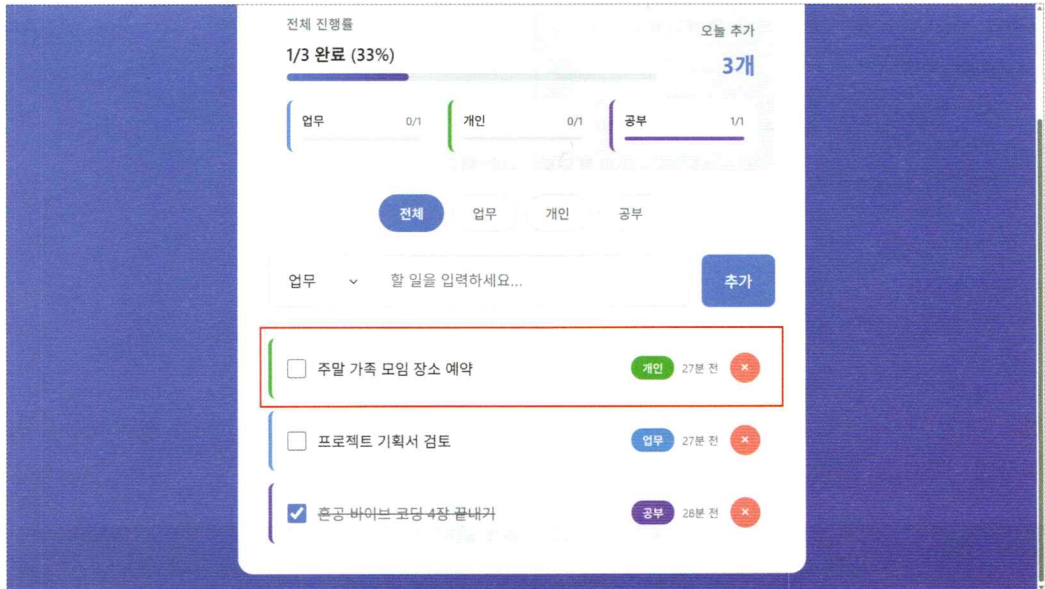
02 완료되었다는 메시지가 보이면 진행 상황을 확인합니다.



03 할 일 관리 앱의 대시보드에 전체 진행률과 카테고리별 미니 진행률 등이 추가되었습니다.



04 할 일 목록에 있는 항목을 더블 클릭하면 수정할 수 있습니다. 수정 기능이 잘 작동하는지 테스트하세요.



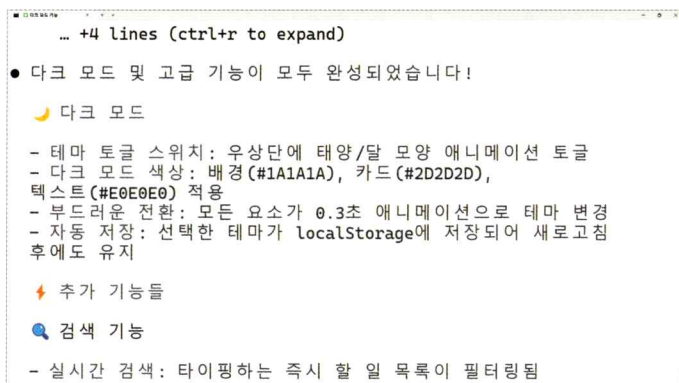
4단계: 다크 모드 및 고급 기능

01 단계별 프롬프트의 네 번째 프롬프트(Step 4)를 복사해 클로드 코드에 입력합니다. 다크 모드를 구현하고 완료된 항목을 삭제하거나 단축키 사용 등의 기능도 추가될 것입니다.

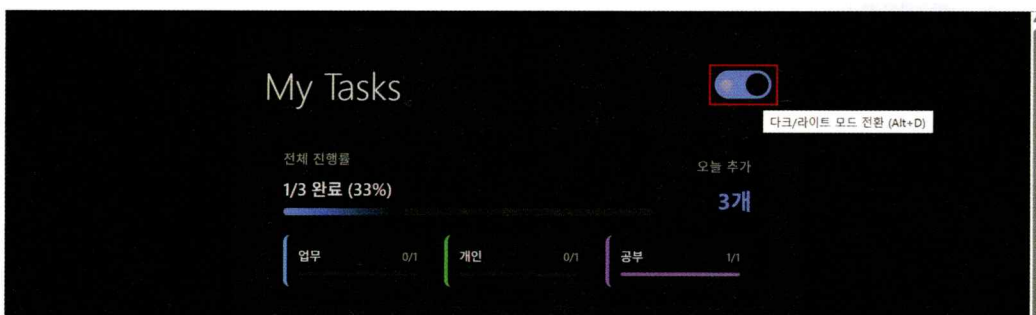
```
> Step 4: 다크 모드 및 고급 기능
다크 모드와 추가 기능들을 구현해줘.

요구사항:
1. 다크 모드:
  - 우측 상단에 다크/라이트 모드 토글 스위치
  - 다크 모드 색상: 배경(#1A1A1A), 카드(#2D2D2D),
    텍스트(#E0E0E0)
  - 선택한 테마 localStorage에 저장
  - 부드러운 전환 애니메이션
2. 추가 기능:
  - "완료된 항목 모두 삭제" 버튼 (확인 다이얼로그 포함)
  - 할 일 검색 기능 (실시간 필터링)
  - 남은 할 일 개수 배지 표시
  - 빈 상태 메시지 ("할 일이 없습니다. 추가해보세요!")
3. 키보드 단축키:
  - Alt+N: 새 할 일 입력창 포커스
  - Alt+1,2,3,4: 카테고리 필터 전환
```

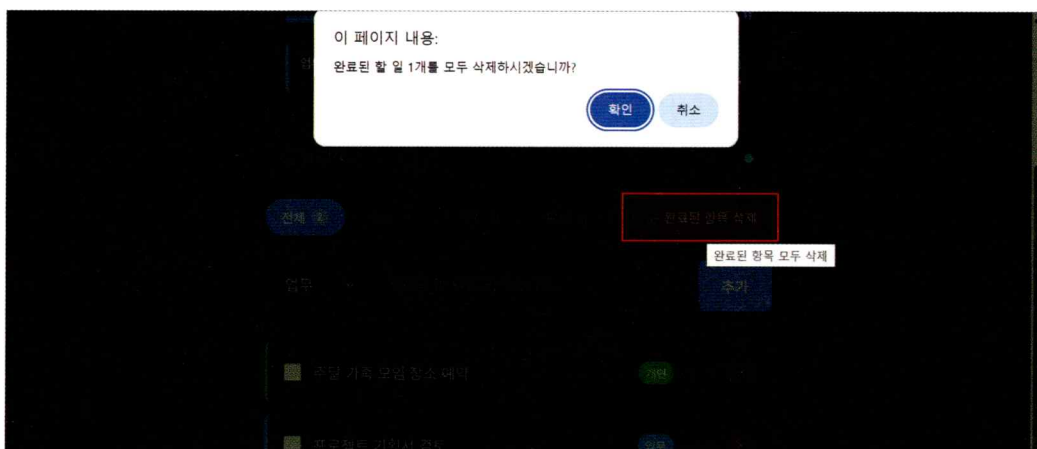
02 완료되었다는 메시지를 확인한 뒤, 추가된 기능을 확인하세요.



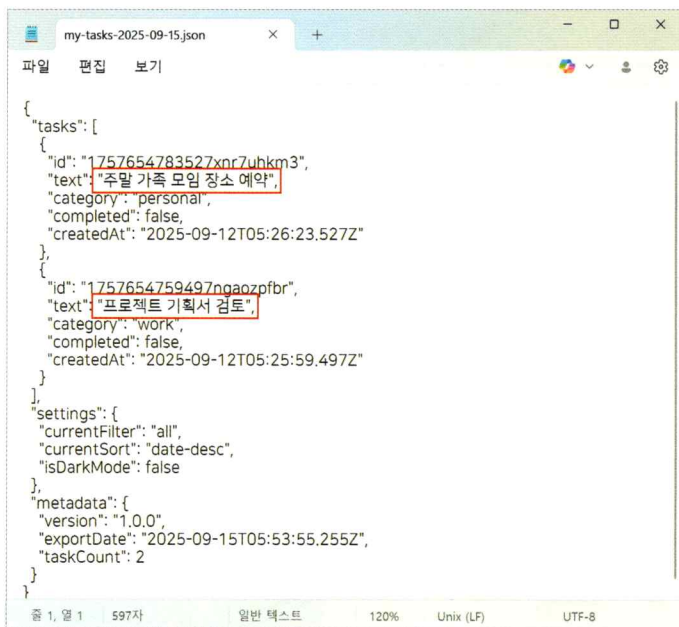
03 할 일 관리 앱에 다크 모드를 선택하는 기능이 생긴 것을 확인합니다. 다크 모드 선택 버튼을 클릭하면 다크 모드가 적용되는 것을 확인할 수 있습니다.



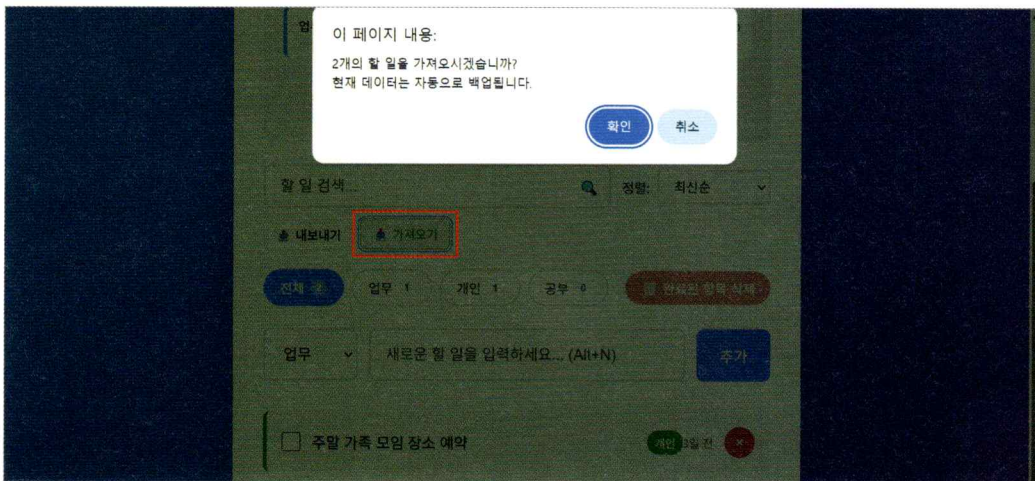
04 완료 항목 일괄 삭제 기능도 테스트해 봅니다. [완료된 항목 삭제] 버튼을 클릭하면 체크 표시한 할일이 모두 삭제됩니다.



04 확장자가 '.json'인 파일을 메모장에서 열어 보세요. 할 일 관리 앱에 저장했던 할 일 목록이 저장되어 있습니다.



05 웹 브라우저에서 실행 중인 할 일 관리 앱에서 [가져오기] 버튼을 클릭하면 해당 JSON 파일을 불러올 수 있습니다. 그러면 모든 할 일 목록이 복원됩니다.



이번 절에서는 클로드 웹과 클로드 코드를 연계하여 프로젝트를 시작하는 과정을 익혔습니다. 2장에서 배운 PRD 작성법이 실제 개발에 어떻게 적용되는지 확인했고, 클로드 코드에서 실제로 앱을 구현했죠. 다음 절에서는 프로젝트를 발전시켜 나가며 더 많은 명령어를 활용해 보겠습니다.

프로젝트를 시작할 때 /init 외에도 알아두면 편리한 명령어들이 있습니다. 현재 세션의 상태를 점검하거나, 사용 중인 모델을 확인하고, 필요한 도움말을 바로 확인할 때 유용하게 쓸 수 있습니다.

/status

/status 명령어는 현재 세션의 상태를 확인할 때 사용합니다. 현재 사용 중인 모델, 토큰 사용량, 남은 컨텍스트 용량 등을 보여줍니다. 긴 작업을 할 때는 주기적으로 상태를 확인하는 것이 좋습니다.

```
> /status

/status
Show Claude Code status including version, model,
account, API connectivity, and tool statuses
```

/model

/model 명령어는 현재 사용 중인 AI 모델을 확인하고 선택할 수 있습니다.

```
> /model [model]

/model
Set the AI model for Claude Code
```

/model 명령어를 실행하면 AI 모델을 선택할 수 있습니다. Max 플랜에서는 Sonnet 모델과 복잡한 작업을 위한 Opus 모델 중에서 선택할 수 있습니다.

```
Select Model
Switch between Claude models. Applies to this session and
future Claude Code sessions. For custom model names, specify
with --model.

> 1. Sonnet Sonnet 4 for daily use ✓

Want Opus 4.1? Run /upgrade to upgrade to Max

Enter to confirm · Esc to exit
```

/help

/help 명령어를 입력하면 사용 가능한 모든 명령어와 설명을 확인할 수 있습니다. 작업 중 새로운 기능이 필요하거나 명령어가 기억나지 않을 때 유용합니다.

```
> /help
/help
Show help and available commands
```

별책 부록에서 명령어 모음을
볼 수 있습니다.



▶ 5가지 키워드로 끝내는 핵심 포인트

- **단계별 프롬프트 전략**이란 PRD를 기반으로 프로젝트를 여러 단계로 나누어 각 단계마다 AI에게 구체적인 목표를 제시함으로써 체계적이고 예측 가능한 결과를 이끌어내는 프롬프트 설계 방식입니다.
- 클로드 코드에서 프로젝트를 시작할 때 필요한 명령어들은 다음과 같습니다.
 - **/init**: 프로젝트 초기화 및 CLAUDE.md 생성
 - **/status**: 현재 작업 상태 확인
 - **/model**: AI 모델 선택 및 변경
 - **/help**: 사용 가능한 명령어 확인

▶ 확인 문제

1. 다음은 클로드 코드에서 프로젝트 초기 설정을 위해 자주 쓰는 명령어에 대한 설명입니다. 빈칸에 알맞은 명령어들을 채워 보세요.

설명	명령어
프로젝트 구조를 분석하고 CLAUDE.md 파일을 자동 생성	
현재 세션의 토큰 사용량과 남은 컨텍스트 용량 확인	
사용할 AI 모델을 선택하거나 Default 옵션으로 변경	
사용 가능한 모든 명령어 목록과 설명을 확인	

2. 다음 설명을 보고 알맞은 용어를 쓰세요.

- ① 프로젝트 폴더 구조를 분석하여 자동으로 생성되는, 프로젝트의 컨텍스트와 규칙을 담은 마크다운 파일 → ()
- ② 제품의 목적, 대상 사용자, 주요 기능, 기술 스펙 등을 체계적으로 정리한 제품 요구사항 문서 → ()
- ③ 할 일 관리 앱에서 데이터를 파일로 저장하고 다시 불러올 때 사용하는 데이터 형식 → ()



04-2

작업 재개와 효율성 높이기

핵심 키워드

세션 관리

--resume

자동 승인 모드

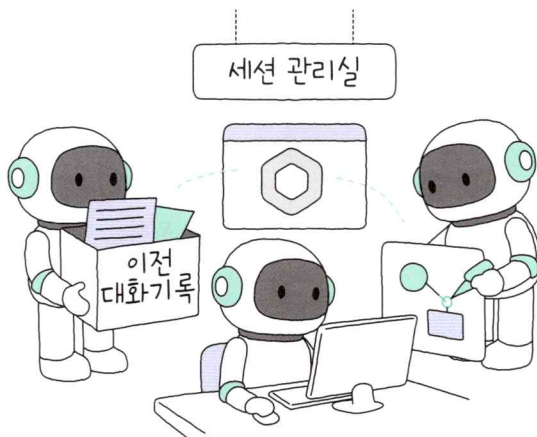
문맥

/compact

할 일 관리 앱을 이어 개발하면서 클로드 코드의 세션 관리와 컨텍스트 최적화 기능을 익힙니다. 특히 --resume 명령어로 대화를 불러오는 방법, **[Shift] + [Tab]** 키로 자동 승인 모드를 활용하는 방법, /context와 /compact 명령어로 문맥을 관리하는 방법 등 장기 프로젝트 개발에 필수적인 기능들을 실습합니다.

시작하기 전에

세션^{session}은 컴퓨터나 프로그램에서 사용자와 시스템이 연결되어 있는 일정한 기간을 말합니다. 클로드 코드에서는 대화를 시작해 명령을 주고받는 동안 유지되는 상태를 말합니다. 클로드 코드는 터미널을 닫으면 이전 대화가 사라지는 것처럼 보이기 때문에, 장기 프로젝트를 진행할 때는 세션과 컨텍스트 관리에 특히 주의해야 합니다.



기능 추가하기

04-1절에서 할 일 관리 앱의 전체 구조와 기능을 구현했습니다. 하지만 실제 개발에서는 한 번의 작업만으로 프로젝트를 완성하기는 어렵죠. 터미널을 닫았다가 다시 열거나, 며칠 후 이어서 작업해야 할 때도 많습니다.

클로드 코드는 터미널을 닫으면 대화가 사라진 것처럼 보이지만, 실제로는 모든 대화 내용이 프로젝트 폴더에 자동 저장됩니다. 이번 절에서는 이렇게 저장된 세션을 다시 불러와, 이전에 하던 작업을 그대로 이어 진행하는 방법을 알아보겠습니다.

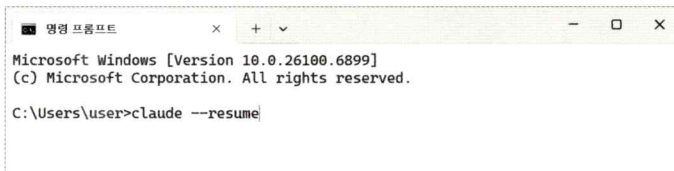
파일 탐색기에서 'Study-02' 폴더를 열고, 마우스 오른쪽 버튼을 눌러 [터미널에서 열기]를 선택해 명령 프롬프트를 실행하세요.

세션 재개하기

01 명령 프롬프트에 'claude --resume' 명령어를 입력합니다.

```
▶ claude --resume
```

'claude --resume'은 이전 대화 기록을 불러와서 마지막 상태 그대로 이어 작업해줄 수 있게 하는 명령어입니다. '--resume' 옵션은 저장된 세션 파일을 읽어와 이전 컨텍스트를 복원합니다.



02 현재 프로젝트의 대화 목록을 보여줍니다. 화살표 키로 원하는 대화를 찾고, [Enter] 키를 눌러 재개합니다.



03 할 일 관리 앱을 구현했던 대화를 찾아 선택한 뒤, `[Enter]` 키를 누릅니다. 대화가 중단된 시점부터 세션이 복원되어, 이어서 작업할 수 있습니다.

```

L (No content)

• Task manager application is now complete and launched! All features have been successfully implemented and tested:

✓ Core Features: Add, edit, delete, toggle tasks with localStorage persistence
✓ Category System: Work, personal, study categories with filtering and color coding
✓ Progress Dashboard: Real-time statistics and progress tracking
✓ Dark Mode: Smooth theme switching with CSS custom properties
✓ Advanced Search: Real-time filtering with debounced performance
✓ Sorting Options: Date, category, completion status, and drag-and-drop manual sorting
✓ Data Management: Export/import functionality with JSON validation
✓ Accessibility: Full ARIA support, keyboard navigation, screen reader compatibility
✓ Performance: Debounced operations, efficient DOM updates, error handling

```

+ 여기서 잠깐 세션 재개 명령어

클로드 코드는 프로젝트별로 대화 세션을 저장해, 언제든 이전 작업을 이어서 진행할 수 있습니다. 만약 작업 중 터미널이 닫히거나, 중간에 다른 명령어를 실행하느라 대화가 끊긴 경우에도 걱정할 필요가 없습니다. 이전 대화를 그대로 불러와 이어서 진행할 수 있는 명령어들이 준비되어 있기 때문입니다.

① `/resume`

먼저 `claude`를 입력해서 클로드 코드를 실행한 후 `/resume` 명령어를 입력하여 이전 대화를 선택하여 재개하는 방법도 있습니다.

```

> /resume

/resume
Resume a conversation

```

② `--continue` 또는 `-c` 옵션

`--continue`는 가장 최근의 마지막 대화를 자동으로 복구하게 해 주는 플래그입니다. 작업 중 연결이 끊기거나 터미널이 종료되었을 때 `--continue` 또는 `-c` 옵션을 사용할 수도 있습니다.

```

Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>claude --continue

```

자동 승인 모드로 작업 효율성 높이기

01 클로드 코드에 다음과 같이 프롬프트를 입력하세요.

프롬프트

현재 할 일 관리 앱에 키워드 기반 자동 카테고리 분류 기능을 추가해 줘.

```
> 현재 할 일 관리 앱에 키워드 기반 자동 카테고리 분류 기능을  
  추가해 줘.█
```

02 그러면 클로드 코드가 대략 다음과 같은 작업을 진행하게 됩니다.

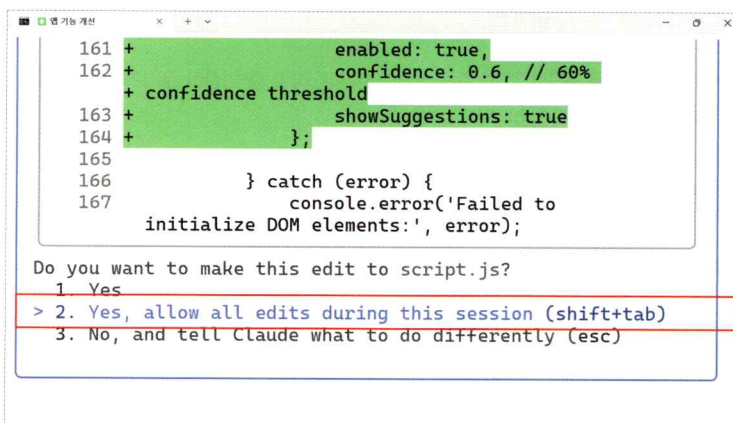
- [] 키워드 기반 자동 분류 시스템 추가
- [] 카테고리를 자동으로 제안하는 UI 업데이트
- [] 다양한 키워드로 자동 분류 기능 테스트

```
* Adding keyword-based auto-categorization system...  
L [ ] Add keyword-based auto-categorization system  
  [ ] Update UI to show auto-category suggestions  
  [ ] Test auto-categorization with various keywords
```

```
>
```

```
▶▶ accept edits on (shift+tab to cycle)
```

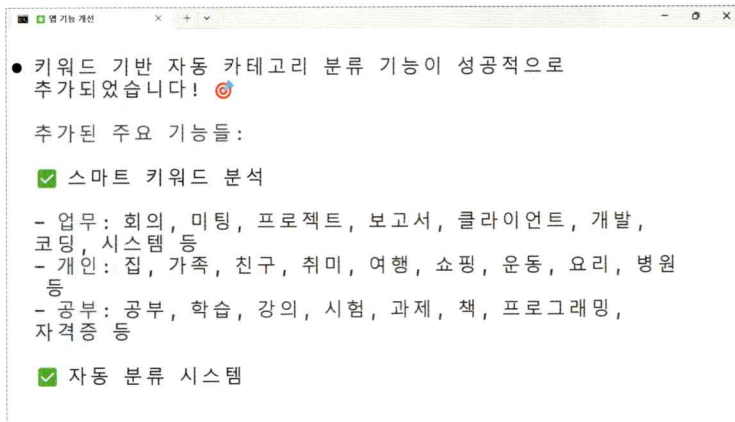
03 **[Shift] + [Tab]** 키를 눌러 '2. Yes, allow all edits during this session'을 선택합니다.



```
161 +         enabled: true,  
162 +         confidence: 0.6, // 60%  
    + confidence threshold  
163 +         showSuggestions: true  
164 +     };  
165  
166     } catch (error) {  
167         console.error('Failed to  
          initialize DOM elements:', error);  
  
Do you want to make this edit to script.js?  
1 Yes  
> 2. Yes, allow all edits during this session (shift+tab)  
3. No, and tell Claude what to do differently (esc)
```

note 하단에 ▶▶ accept edits on (shift+tab to cycle)이 표시되는 동안에는 모든 수정을 자동으로 승인하게 되며, 다시 **[Shift] + [Tab]** 키를 누르면 일반 모드로 돌아옵니다.

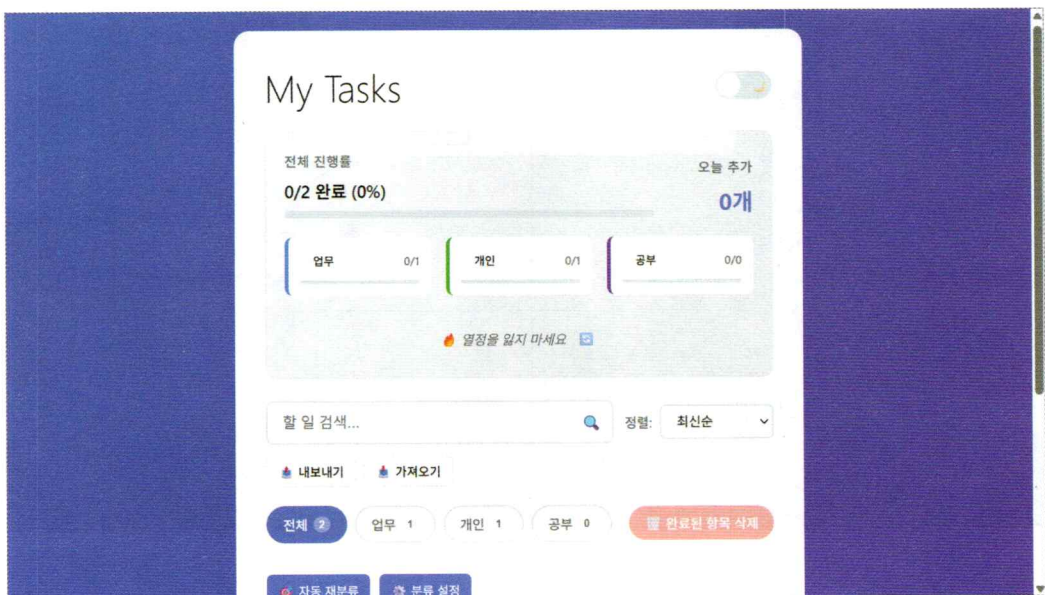
04 작업이 완료되면 기능이 성공적으로 추가되었다는 메시지를 볼 수 있습니다.



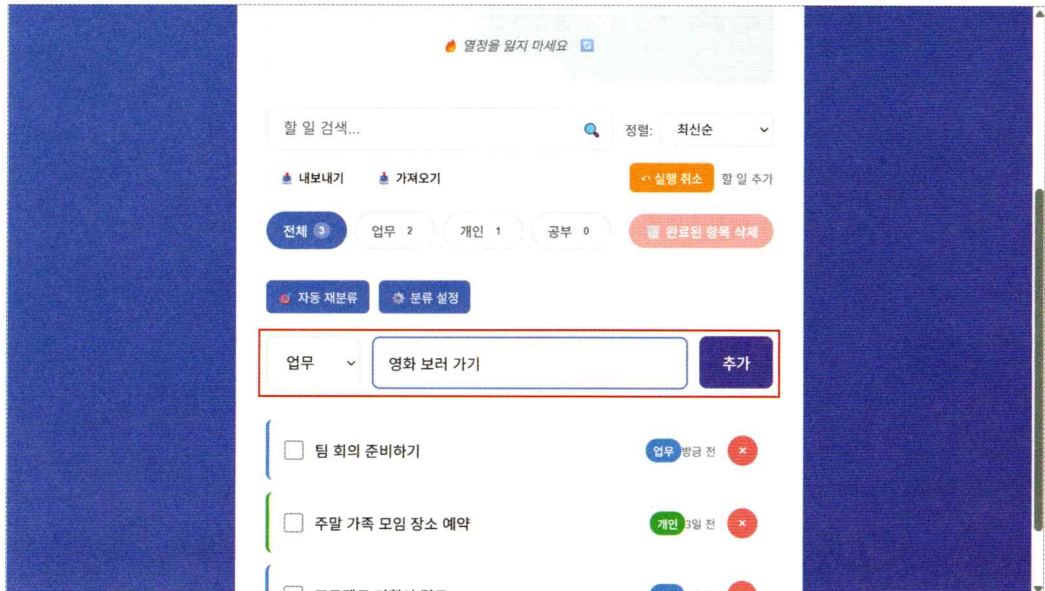
기능 점검하기

요청한 대로 할 일을 추가하면 앱이 자동으로 카테고리를 분류해 주는지 확인해 보시다. 요청한 기능 외 세부 사항은 개인마다 다를 수 있습니다. 클로드 코드의 안내에 따라 구현된 기능이 제대로 작동하는지 확인해 보세요. 이 책에서는 ‘스마트 키워드 분석’, ‘자동 분류 시스템’ 기능이 추가되었네요.

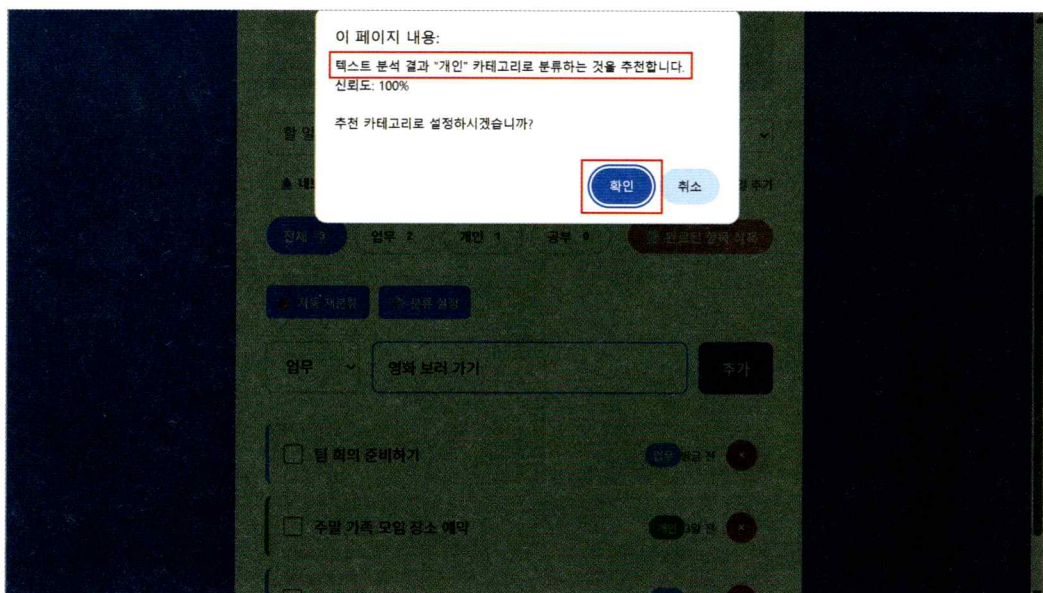
01 변경 사항이 잘 반영되었는지 확인하기 위해 데스크톱 파일 탐색기의 ‘Study-02’ 폴더에서 ‘index.html’ 파일을 실행합니다.



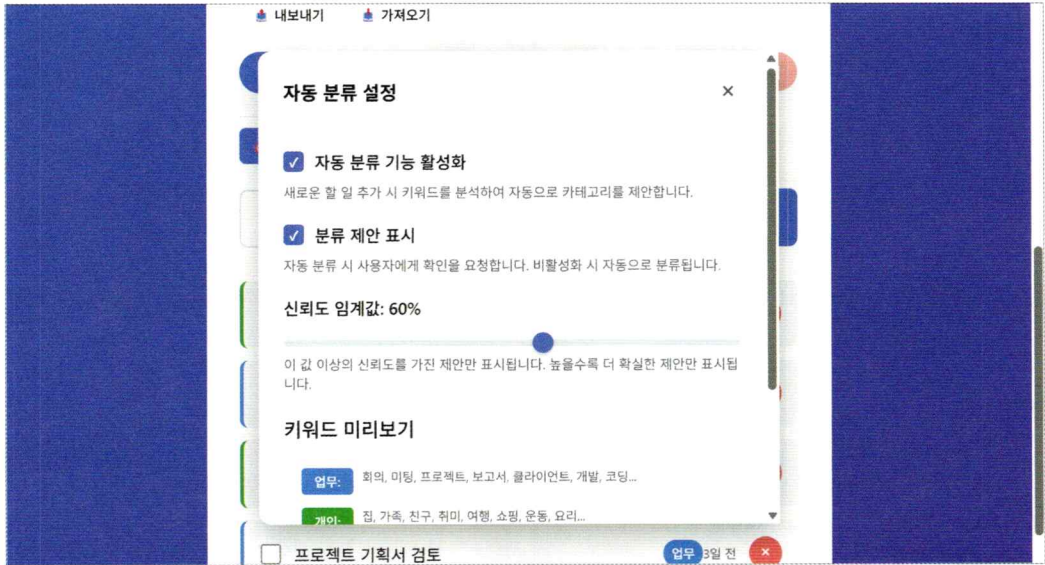
02 카테고리를 자동으로 분류하는지 확인하기 위해 [할 일 검색...] 창에 '영화 보러 가기'라고 입력하고, 카테고리는 '업무'로 둔 뒤 [추가] 버튼을 눌러 보겠습니다.



03 할 일 관리 앱이 '영화 보러 가기'의 카테고리를 '개인'으로 분류하는 것을 추천합니다. [확인]을 눌러 진행하면 할 일이 '개인' 카테고리로 분류되어 정상적으로 등록됩니다.



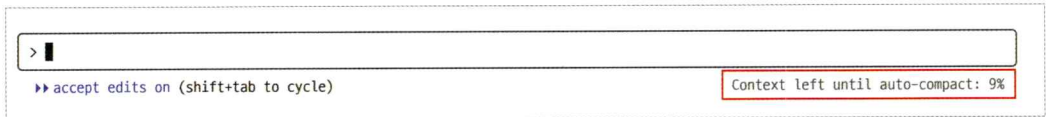
04 할 일 관리 앱의 초기 화면에는 [자동 재분류], [분류 설정] 버튼이 있어서 카테고리 재분류 기능을 활성/비활성화 하거나 세부 옵션을 조정할 수도 있습니다.



컨텍스트 관리하기

클로드 코드에서 작업하다보면 종종 다음과 같은 메시지를 볼 수 있습니다.

자동 압축까지 남은 컨텍스트: 9%



이 메시지의 의미는 현재 대화의 **컨텍스트** context가 자동 압축 auto-compact 실행 임계값에 가까워졌다는 경고입니다. 컨텍스트란 클로드 코드가 현재 세션에서 기억하고 있는 모든 대화 내용과 작업 정보입니다. 즉 이 메시지는 AI가 기억할 수 있는 공간이 부족해지고 있으며, 오래된 대화를 압축하여 토큰 사용량을 줄인 뒤 작업을 계속하겠다는 뜻입니다.

클로드가 자동으로 전체 대화의 일부를 요약하는 과정에서 정보 손실이 있을 수 있습니다. 작업 단위를 끝까지 마무리하고 요약하지 않고, 갑자기 중간 부분에서 억지로 요약하는 경우도 있죠. 따라서 클로드가 컨텍스트를 자동 압축하기 전에 수시로 컨텍스트 상태를 체크하여 적절한 시점에 직접 압축하는 것이 좋습니다.

컨텍스트 상태 확인하기

현재 컨텍스트 상태와 사용량을 확인하는 명령어는 `/context`입니다.

화면 해설

현재 컨텍스트 사용량을 컬러 그리드로 시각화합니다.

```
> /context
/context
Visualize current context usage as a colored grid
```

클로드 코드에 `/context` 명령어를 입력하고, `Enter` 키를 누르세요. 현재까지의 컨텍스트 사용량을 확인할 수 있습니다.

화면 해설

컨텍스트 사용 현황

모델: claude-sonnet-4-20250514 • 현재 사용량: 93k / 200k 토큰 (46%)

시스템 프롬프트: 3.1k 토큰 (1.5%)

시스템 도구: 11.5k 토큰 (5.7%)

메모리 파일: 84 토큰 (0.0%)

메시지: 78.0k 토큰 (39.0%)

여유 공간: 107.4k 토큰 (53.7%)

메모리 파일 • `/memory`

Project: C:\Users\user\Desktop\VibeCoding\Study-02\CLAUDE.md: 84 토큰

```

> /context
└ Context Usage
  claude-sonnet-4-20250514 • 93k/200k tokens (46%)
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️ System prompt: 3.1k tokens (1.5%)
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️ System tools: 11.5k tokens (5.7%)
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️ Memory files: 84 tokens (0.0%)
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️ Messages: 78.0k tokens (39.0%)
  🗂️ 🗂️ 🗂️ 🗂️ 🗂️ Free space: 107.4k (53.7%)

  memory files • /memory
  Project (C:\Users\user\Desktop\VibeCoding\Study-02\CLAUDE.md): 84
  tokens

>
  >> accept edits on (shift+tab to cycle)
```

note 토큰 한도는 내 구독 플랜에 따라 다릅니다.

컨텍스트 압축하기

이제 컨텍스트 관리를 위해 수동으로 압축해 보겠습니다. 현재까지의 대화 내용을 요약하는 압축 명령어는 `/compact`입니다.

화면 해설

대화 기록을 지우고, 요약본은 남겨둡니다.

옵션: `/compact` [요약 지침]

```
> /compact <optional custom summarization instructions>

/compact
Clear conversation history but keep a summary in context.
Optional: /compact [instructions for summarization]
```

`/compact` 명령어는 사용자의 필요에 따라 컨텍스트의 압축 수준과 범위를 세밀하게 조정할 수 있는 고급 옵션을 제공합니다. 단계별로 1~3 레벨이 있으며, `/compact` 명령어 뒤에 공백을 한 칸 두고 옵션을 추가로 입력하면 됩니다.

- `/compact --level 1`: 불필요한 공백이나 중복을 제거합니다.
- `/compact --level 2`: 변수명을 단축하고 구조를 단순화합니다.
- `/compact --level 3`: 핵심 로직만 유지하고 주석, 변수명 등을 최대한 압축합니다.

01 컨텍스트를 압축하기 위해 클로드 코드에 `/compact` 명령어를 입력하세요. 압축이 끝나면 'Compacted (ctrl+o to see full summary)'라는 메시지가 표시되고, 화면 하단의 `auto-compact` 경고 메시지가 사라집니다.

```
> /compact
Compacted (ctrl+o to see full summary) ← 압축되었습니다. 전체 요약을 보려면 Ctrl + O 키를 누르세요.
Read ...\.claude\todos\9db493c7-cc5f-48d3-8d24-4e0f07e9cd67-agent-9db493
c7-cc5f-
48d3-8d24-4e0f07e9cd67.json (1 lines)
```

```
> 
? for shortcuts
```

02 `Ctrl` + `O` 키를 누르면 클로드가 중요한 작업 내용을 제대로 보존했는지 확인할 수 있도록 압축 과정에서 생성된 전체 요약을 볼 수 있습니다.

▶ 4가지 키워드로 끝내는 핵심 포인트

- `--resume` 명령어는 대화 목록에서 원하는 세션을 선택해 복원합니다.
- `[Shift] + [Tab]` 키를 누르면 **자동 승인 모드**로 전환되어 반복적인 권한 승인 요청을 생략할 수 있습니다.
- **컨텍스트**는 현재 대화와 작업 정보를 저장 및 표시한 메모리 공간입니다.
- `/compact` 명령어는 대화 기록을 요약해 컨텍스트 공간을 확보합니다.

▶ 확인 문제

1. 다음 설명 중 틀린 것을 고르세요.

- ① `/compact`는 대화 내용을 요약하여 토큰을 절약한다.
- ② `auto-compact`는 컨텍스트가 임계값에 도달하면 자동으로 실행된다.
- ③ 압축 후에도 `CLAUDE.md`와 프로젝트 파일은 유지된다.
- ④ `[Ctrl] + [R]`은 `/context` 명령어의 단축키이다.
- ⑤ “Accept all” 모드는 `[Shift] + [Tab]`으로 전환할 수 있다.

2. 다음 상황에서 사용할 적절한 명령어나 옵션을 쓰세요.

상황	명령어 / 옵션
여러 대화 중 특정 대화를 선택해서 재개하고 싶을 때	
가장 최근 대화를 즉시 이어서 작업하고 싶을 때	
현재 컨텍스트 사용량을 시각적으로 확인하고 싶을 때	
대화가 길어져서 압축이 필요할 때	
파일 수정 승인을 매번 누르기 번거로울 때	

04-3

프로젝트 개선과 작업 관리

핵심 키워드

캡처

@ 기호

/config

/doctor

/export

할 일 관리 앱을 데스크톱 버전으로 확장하면서 클로드 코드의 프로젝트 관리 기능들을 익힙니다. 클로드 코드에 이미지를 전달해서 요구사항을 명시하고, @ 기호로 파일을 빠르게 분석하며, 설정과 진단 명령어로 작업 환경을 점검합니다. 마지막으로 전체 작업 과정을 문서화하는 방법까지 실습합니다.

시작하기 전에

하나의 AI가 여러 종류의 정보를 함께 이해하고 처리하는 AI를 **멀티모달 AI** multimodal AI 이라고 합니다. 멀티모달 AI는 텍스트나 코드뿐만 아니라 이미지, 음성 등 서로 다른 형태의 데이터를 함께 인식할 수 있습니다.

클로드 코드가 실행될 때 백엔드 back-end에서는 Sonnet이나 Opus 같은 클로드의 멀티모달 AI 모델이 동작합니다. 클로드 코드는 명령어 기반의 인터페이스지만, 이미지를 전달하면 내부 모델이 그 이미지를 분석해 텍스트로 설명하거나 구조를 파악한 뒤 코드로 바꿔 줄 수 있습니다.

캡처 이미지로 데스크톱 버전 앱의 UI 개선하기

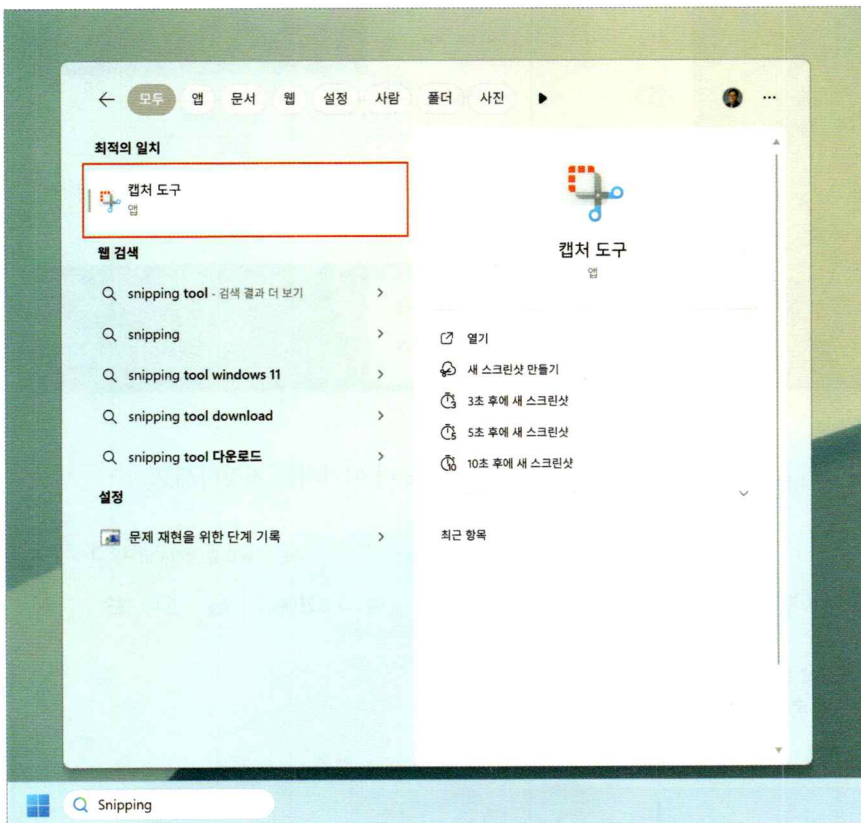
백문이 불여일견이라는 말은 클로드 코드와 작업할 때도 통합니다. 장황한 설명보다는 한 번 보여주는 편이 훨씬 효과적이죠. 화면을 직접 **캡처**^{capture}해 전달하면 클로드 코드가 요구사항을 더 정확하고 빠르게 이해할 수 있습니다.

현재 결과가 모바일 중심의 디자인이라 화면에 꽉차게 나오는 PC 버전으로 만들어 달라고 하였습니다. 이전 결과를 보관하기 위해 'web_version'이라는 폴더를 새로 만들어 그곳에 저장해 달라고 하였습니다.

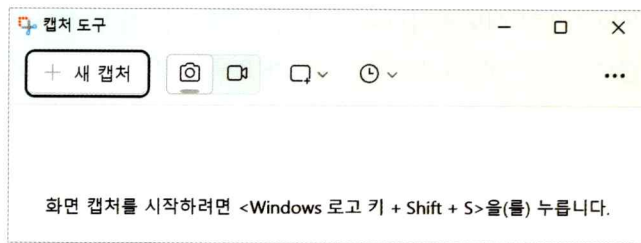
이제 현재 작업한 할 일 관리 앱을 캡처해 클로드 코드에 전달해 보겠습니다.

클로드 코드에 이미지 전달하기

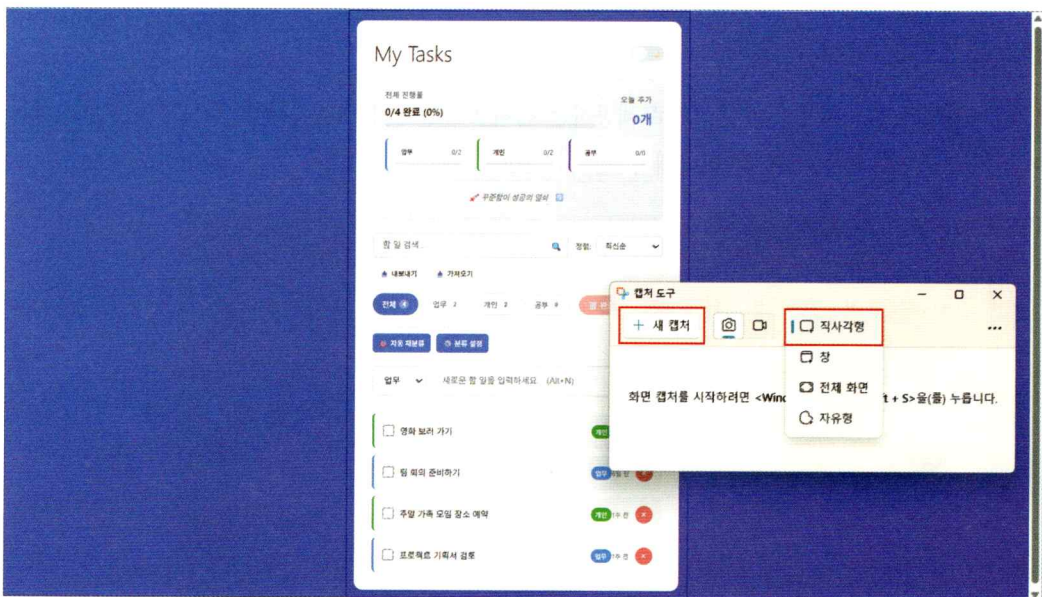
01 여기서는 윈도우에서 기본적으로 제공하는 캡처 도구를 사용하겠습니다. 윈도우의 [시작] 메뉴에서 'Snipping' 혹은 '캡처'를 입력해서 [캡처 도구]를 실행합니다.



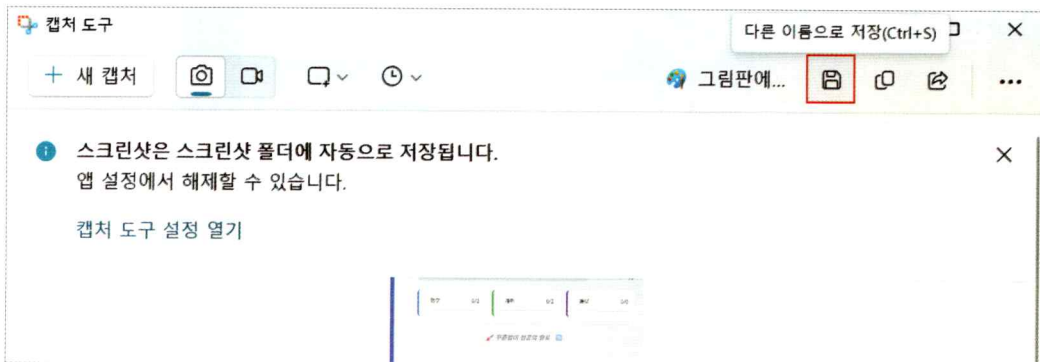
02 캡처 도구를 실행하면 [+ 새 캡처] 버튼을 클릭하여 캡처를 시작할 수 있습니다.



03 할 일 관리 앱을 실행하고 캡처 도구의 [직사각형] 옵션을 선택한 뒤, [+ 새 캡처] 버튼을 누르고 앱의 메인 화면을 드래그해서 캡처합니다.



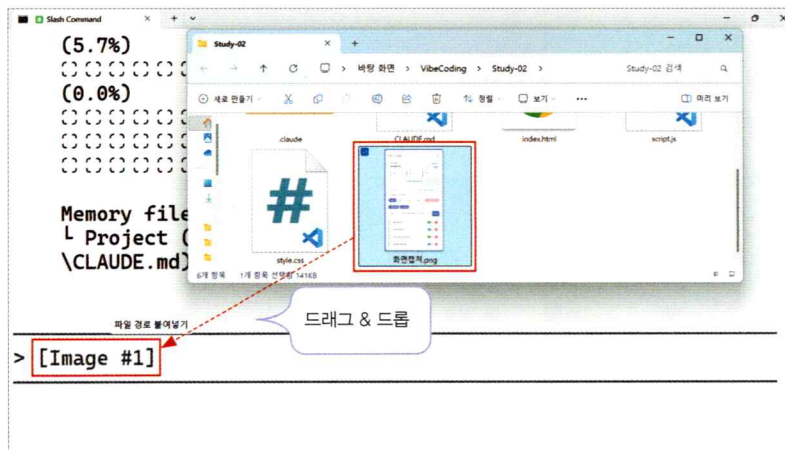
04 [캡처 도구] 창에서 [다른 이름으로 저장(⌘S)] 아이콘을 눌러 이미지를 저장하세요.



05 이미지를 'Study-02' 폴더에 '화면캡처.png'라는 이름으로 저장하겠습니다. [저장] 버튼을 클릭하세요.



06 클로드 코드에 이미지를 입력하는 방법은 매우 간단합니다. '화면캡처.png' 파일을 드래그해서 클로드 코드가 실행 중인 명령 프롬프트 창에 드롭하는 것입니다. 이미지가 정상적으로 입력되면 프롬프트 입력창에 '[Image #1]' 메시지가 표시됩니다.



note 이미지의 파일 경로를 클로드 코드에 직접 입력해도 동일한 작업이 가능합니다.

UI 개선하기

01 이제 디자인 개선을 위해 요청 사항을 입력해 보겠습니다. 이미지를 전달한 입력창에 다음과 같이 프롬프트를 작성하세요.

프롬프트

[Image #1] 현재 디자인은 모바일에 최적화되어 있는 것 같아. 데스크톱 환경에서도 전체 화면에서 UI를 볼 수 있도록 디자인을 수정해 줘. 수정한 디자인은 'web_version'이라는 새로운 폴더를 생성한 뒤, 그곳에 저장해 줘.

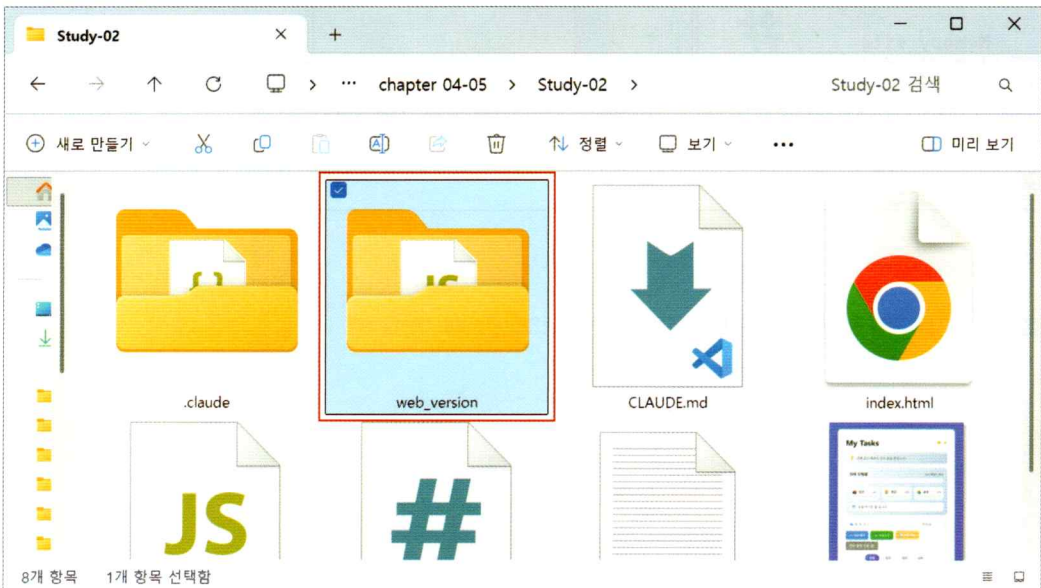
```
Slack Command
Messages: 1.9k tokens (1.0%)
Free space: 183.4k (91.7%)

Memory files · /memory
└ Project (C:\Users\user\Desktop\VibeCoding\Study-02\CLAUDE.md): 84 tokens

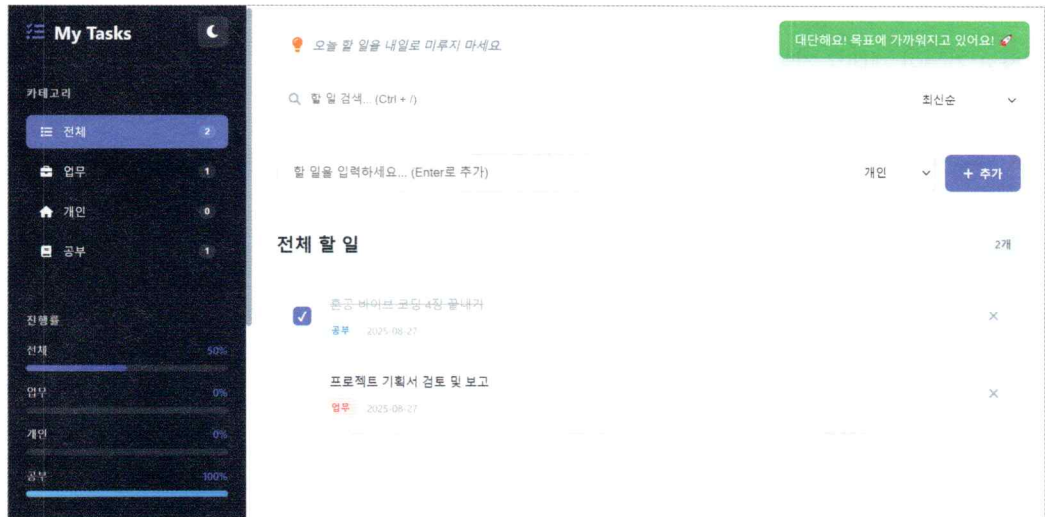
> [Image #1] 현재 디자인은 모바일에 최적화되어 있는 것
  같아. 데스크톱 환경에서도 전체 화면에서 UI를 볼 수
  있도록 디자인을 수정해 줘. 수정한 디자인은
  'web_version'이라는 새로운 폴더를 생성한 뒤, 그곳에
  저장해 줘.
```

note 권한이 필요한 작업을 만날 때마다 이 작업을 실행할 것인지 묻습니다. '2. Yes, and don't ask again for mkdir..'을 선택하여 작업을 자동으로 이어가세요.

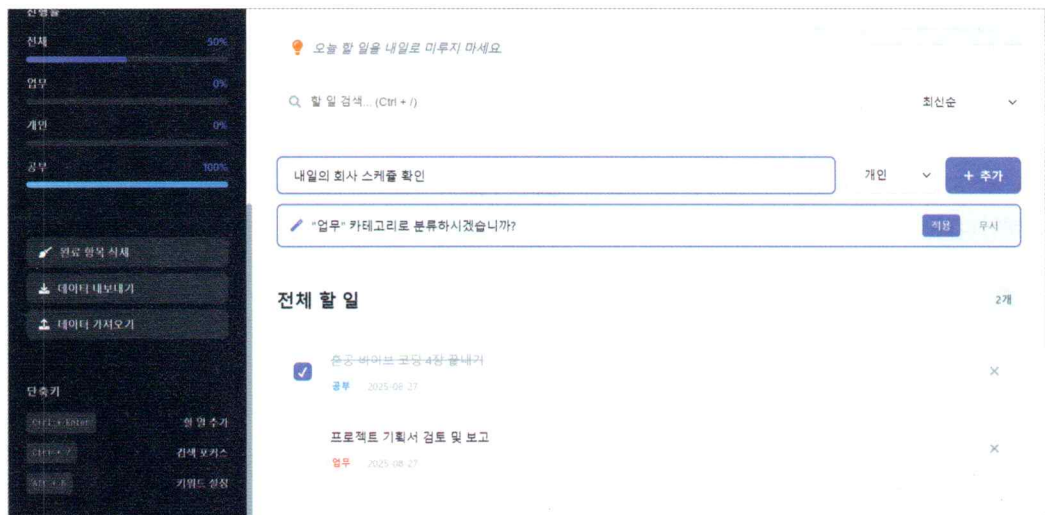
02 실행이 끝나면 실행 결과를 확인하라는 메시지가 나타납니다. 'Study-02' 폴더에 'web_version' 폴더가 생성되었는지 확인하세요.



03 'web_version' 폴더의 'index.html' 파일을 더블 클릭하여 실행하면, 데스크톱 환경에 최적화된 할 일 관리 앱을 확인할 수 있습니다.

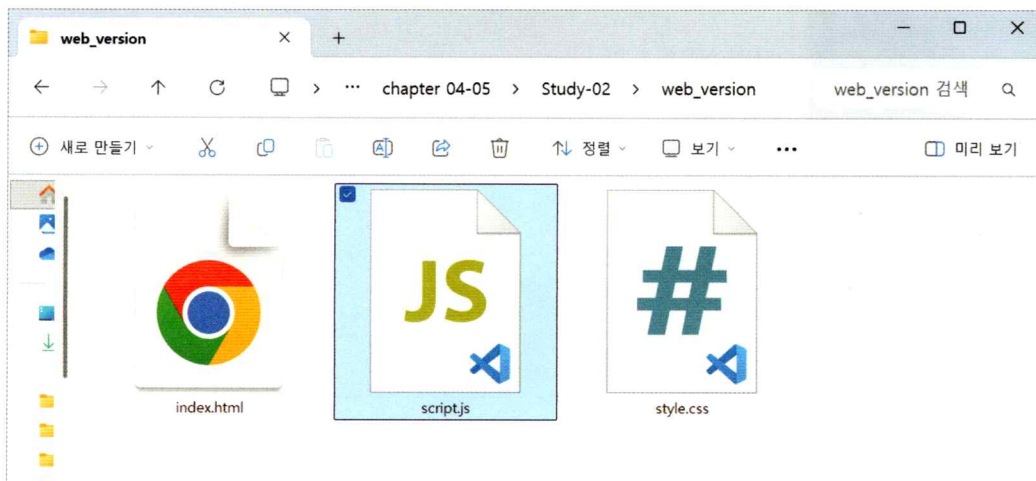


04 모든 기능이 정상적으로 작동하는지 확인하세요. 할 일 입력란에 '내일 회사 스케줄 확인'이라고 입력하면 자동으로 '업무' 카테고리로 분류합니다.



참조를 통해 코드 빠르게 점검하기

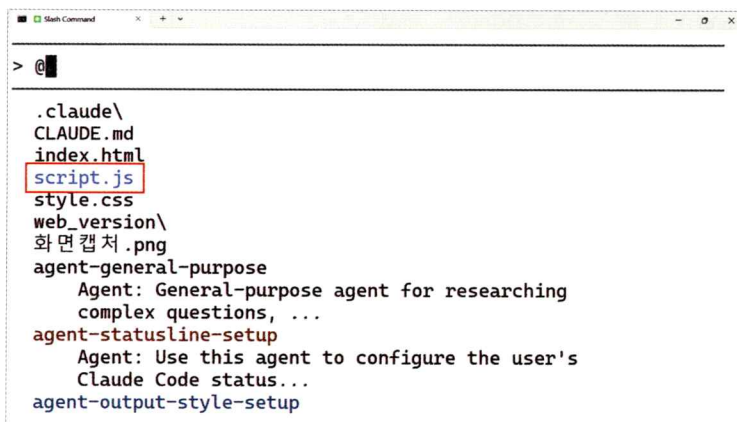
클로드 코드의 @ 기호를 활용하면 생성된 코드를 빠르게 검토하고 분석할 수 있습니다. @ 기호를 사용해 작업 폴더에 생성된 'script.js'를 불러와, 앞서 만든 할 일 관리 앱의 자바스크립트 코드를 분석하고 기능이 제대로 작동하는지 점검해 보겠습니다.



@ 기호로 코드 분석하기

@ 기호를 사용하면 클로드 코드에 파일이나 폴더를 즉시 불러올 수 있어 작업 속도가 향상됩니다.

01 클로드 코드에 '@'를 입력하면 불러올 수 있는 파일 목록을 보여줍니다. 이 중 'script.js' 파일을 선택합니다.



note 특정 파일을 찾아달라고 요청할 수도 있지만, 검색 시간이 필요해 작업이 지연될 수 있습니다.

02 자바스크립트 파일에서 다크 모드 기능이 잘 작성되어 있는지 확인하기 위해, 다음과 같이 프롬프트를 입력합니다. 그러면 클로드 코드가 'script.js' 파일 전체를 읽고 즉시 분석합니다.

프롬프트

@script.js 파일에서 다크 모드 관련 함수를 찾고 설명해 줘.

```
> @script.js 파일에서 다크 모드 관련 함수를 찾고  
설명해 줘. █
```

```
▶▶ accept edits on (shift+tab to cycle)
```

03 파일 분석이 끝나면 요청한 대로 함수의 이름과 기능, 위치를 알려 주고, 단축키와 같은 숨겨진 기능도 소개합니다.

```
L Found 147 lines (ctrl+o to expand)

● script.js 파일에서 다크 모드 관련 함수들을  
찾았습니다. 주요 함수들과 기능을  
설명해드리겠습니다:

1. toggleTheme() 함수 (564-577줄)

toggleTheme() {
  try {
    this.isDarkMode = !this.isDarkMode;
    this.applyTheme();
    this.debounceSave();

    const themeText = this.isDarkMode ? '다크  
모드' : '라이트 모드';
    this.showFeedback(`${themeText}로
```

이처럼 @ 기호를 사용하면 파일 전체를 훑어보며 필요한 정보를 찾는 수고를 덜 수 있습니다. 1,000 줄이 넘는 코드에서 다크 모드 함수를 정확히 찾아내는 작업을 사람이 직접 하면 시간이 오래 걸리지만, 클로드 코드는 순식간에 처리합니다.

@ 기호로 폴더 구조 확인하기

이번에는 방금 만든 'web_version' 폴더를 확인해 보겠습니다.

01 클로드 코드에 '@web_version'을 입력해서 'web_version' 폴더 전체를 불러옵니다. 'web_version' 폴더에 들어 있는 하위 파일 리스트가 보입니다.

```
> @web_version

web_version\
web_version\style.css
web_version\script.js
web_version\index.html
```

02 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인하기 위해 다음과 같이 프롬프트를 입력합니다.

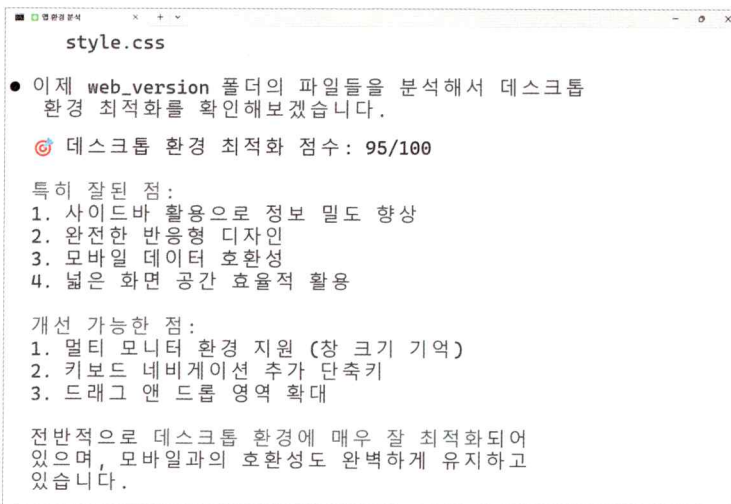
프롬프트

@web_version 폴더에 있는 파일들을 분석해서 할 일 관리 앱이 데스크톱 환경에 최적화되었는지 확인해 줘.

```
> @web_version 폴더에 있는 파일들을 분석해서 할 일
  관리 앱이 데스크톱 환경에 최적화 되었는지 확인해 줘.

>> accept edits on (shift+tab to cycle)
```

03 클로드 코드가 폴더 내 모든 파일을 읽고 종합 분석 결과를 보여줍니다. 내용을 요약하면 대략 다음과 같은 분석 결과가 나타납니다.



이처럼 @ 기호로 폴더를 지정하면 여러 파일을 일일이 열어보지 않아도 전체적인 구조와 특징을 한 눈에 파악할 수 있습니다. 특히 웹 개발처럼 HTML, CSS, 자바스크립트가 서로 연결된 경우에는 클로드 코드가 파일 간의 연관성까지 분석해서 종합적인 평가를 제공합니다.

클로드 코드 프로젝트 관리하기

이번에는 할 일 관리 앱을 만들면서 진행해 온 여러 작업을 마무리하고, 프로젝트를 체계적으로 정리하는 방법을 배워 보겠습니다. 클로드 코드는 작업 환경을 점검하고, 설정을 조정하며, 대화 내용을 보관하는 유용한 명령어들을 제공합니다.

/config 명령어로 설정 확인하기

/config 명령어는 프로그램이나 시스템의 설정(configuration)을 확인하거나 변경할 때 사용합니다. 즉 프로젝트를 어떤 방식으로 다룰지에 대한 규칙을 관리하는 도구라고 할 수 있습니다. 클로드 코드에서는 프로젝트에 적용된 현재 설정값, 예컨대 파일 저장 권한이나 출력 형식 등 정보를 확인할 수 있으며 특정 설정을 원하는 값으로 변경할 수도 있습니다.

```
> /config
/config (theme)
  Open config panel
```

클로드 코드에 '/config' 명령어를 입력한 뒤 **Enter** 키를 누르세요. 다음과 같은 설정 항목을 볼 수 있습니다.

설정 항목	기본값	설명
Auto-compact	TRUE	문맥이 일정 수준 차면 자동으로 /compact를 실행해 대화기록을 요약하고 유지합니다. false로 설정하면 수동으로만 압축할 수 있습니다.
Use todo list	TRUE	클로드 코드가 작업 내용을 할 일 목록 형식으로 관리해 단계별 진행 상황을 추적합니다.
Verbose output	FALSE	출력 결과를 장황하게(verbose) 보여줄지 여부를 결정합니다. TRUE로 설정하면 상세한 디버그 정보가 표시됩니다.
Auto-updates	TRUE	클로드 코드 자동 업데이트를 활성화합니다. 새 버전이 나오면 자동으로 다운로드 및 설치됩니다.
Theme	Light mode	터미널 테마를 설정합니다. Light mode와 Dark mode 중 선택 가능합니다.
Notifications	Auto	작업 완료 시 알림 방식을 설정합니다. Auto, Bell, None 중 선택할 수 있습니다.
Output style	default	클로드의 응답 스타일을 결정합니다. default는 표준 형식입니다.
Editor mode	normal	편집 모드를 설정합니다. normal 또는 vim 모드를 선택할 수 있습니다.
Model	Default (recommended)	사용할 AI 모델을 선택합니다. Default는 작업에 따라 자동으로 최적 모델을 선택합니다.
Auto-connect to IDE	FALSE	IDE와 자동 연결 여부를 설정합니다. VS Code 등과 자동으로 연동할지 결정합니다.

note **↑**, **↓** 키를 이용해 항목을 선택하고 **Enter**/**Tab**/**Space** 키로 값을 변경할 수 있습니다. 설정 화면을 닫으려면 **Esc** 키를 누르세요.

/doctor 명령어로 시스템 진단하기

/doctor 명령어는 이름 그대로 의사가 진찰을 하듯 클로드 코드에서 현재 프로젝트가 정상적으로 실행되는지 환경을 점검하고 문제를 진단합니다.

```
> /doctor
/doctor
Diagnose and verify your Claude Code installation and settings
```

클로드 코드에 '/doctor' 명령어를 입력하고 **[Enter]** 키를 누르면 항목별 진단 결과를 확인할 수 있습니다.

화면 해설

- 현재 실행 중: 현재 실행 중인 클로드 코드의 버전
- 경로: 노드 JS가 설치된 위치
- 호출된 위치: 클로드 코드 실행 파일이 실제로 호출(invoked)된 경로
- 자동 업데이트 가능 여부
- 업데이트 권한
- 코드 검색 기능 정상 작동 여부

```
> /doctor
Diagnostics
├ Currently running: npm-global (1.0.120)
├ Path: C:\Program Files\nodejs\node.exe
├ Invoked: C:\Users\user\AppData\Roaming\npm\node_modules\@anthropic-a
i\claude-code\cli.js
├ Config install method: global
├ Auto-updates enabled: default (true)
├ Update permissions: Yes
└ Search: OK (vendor)
```

만약 어떤 항목에 문제가 있다면 클로드 코드가 해결 방법을 제시해줄 것입니다. 모든 항목이 정상이 라면 **[Enter]** 키를 눌러 진단을 마칩니다.

/export 명령어로 대화 내보내기

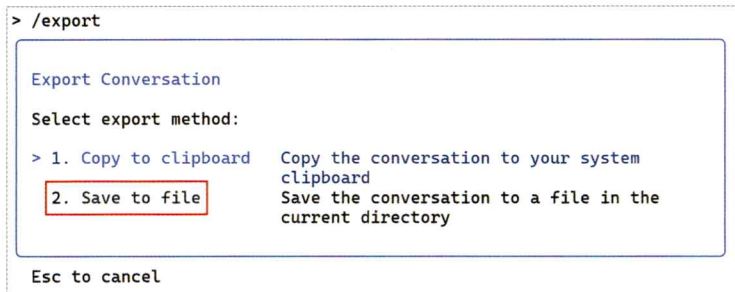
/export 명령어는 현재 작업 중인 프로젝트를 하나의 결과물로 묶어 파일로 내보냅니다. 클로드 코 드에서 작업한 여러 소스 코드, 수정 기록 등을 파일로 보관하면 작업 환경을 옮기거나 협업자에게 공유할 때 좋은 참고 자료로 사용할 수 있습니다. 핵심 대화를 추출해 다른 프로젝트의 CLAUDE.md 파일에 추가할 수도 있습니다.

```
> /export [filename]
/export
Export the current conversation to a file or clipboard
```

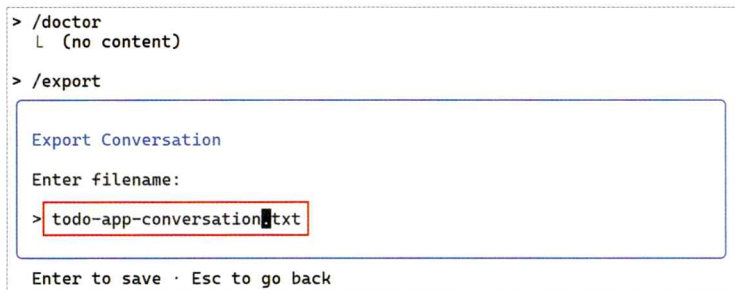
01 클로드 코드에 '/export' 명령어를 입력하고 **[Enter]** 키를 누르세요. 다음과 같이 두 가지 옵션을 선택할 수 있습니다. '2. Save to file'을 선택하고 **[Enter]** 키를 누르세요.

화면 해설

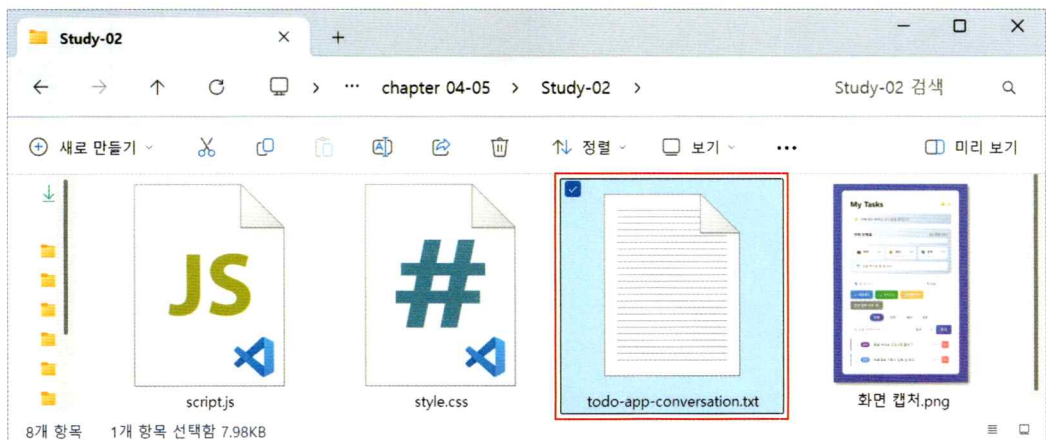
1. 클립보드로 복사: 전체 대화를 복사하여 붙여 넣을 수 있습니다.
2. 파일로 저장: 대화 내용이 파일로 저장됩니다.



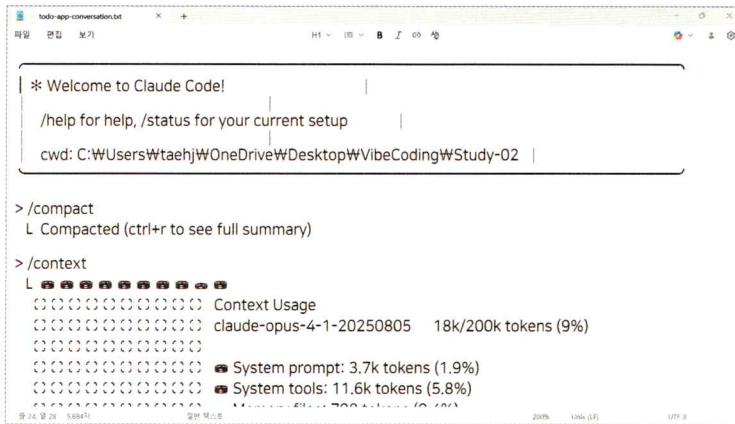
02 파일 이름을 'todo-app-conversation.txt'로 수정하고 **[Enter]** 키를 눌러 저장합니다.



03 데스크톱의 'Study-02' 폴더에 'todo-app-conversation.txt' 파일이 저장되었습니다.



04 메모장을 사용해서 'todo-app-conversation.txt' 파일을 열면 지금까지 클로드 코드에서 작업한 대화 내용이 기록되어 있음을 확인할 수 있습니다.



이번 절에서는 클로드 코드의 프로젝트 개선 및 관리 기능을 종합적으로 실습해 보았습니다. 스크린 캡처로 모바일 중심의 UI를 데스크톱 환경에 맞게 개선했고, @ 기호로 파일과 폴더를 빠르게 분석했으며, /config와 /doctor 명령어로 작업 환경을 점검하고 최적화하는 방법을 배웠습니다. 마지막으로 /export 명령어로 전체 작업 기록을 문서화하여 협업과 향후 프로젝트에 참고할 수 있도록 정리했습니다.

이제 클로드 코드의 다양한 도구를 활용해 프로젝트를 체계적으로 관리할 수 있게 되었습니다. 다음 장에서는 한층 더 복잡한 프로젝트에 도전해 보겠습니다.

마무리

▶ 5가지 키워드로 끝내는 핵심 포인트

- 화면을 직접 **캡처**해 클로드 코드에 전달하면 장황한 설명 없이도 요구사항을 명확하고 빠르게 전달할 수 있습니다.
- **@** 기호를 사용하면 파일 및 폴더를 즉시 참조하고 분석할 수 있습니다.
- **/config** 명령어는 클로드 코드의 프로젝트 환경 설정을 관리할 수 있습니다.
- **/doctor** 명령어는 시스템 상태를 진단해서 버전, 설치 방식, 업데이트 권한 등을 점검합니다.
- **/export** 명령어는 작업 기록을 클립보드나 파일에 저장하고 문서화합니다.

▶ 확인 문제

1. 다음 설명 중 틀린 것을 고르세요.

- ① 스크린 캡처한 이미지는 드래그앤드롭으로 전달할 수 있다.
- ② @ 기호를 사용하면 파일 검색 시간 없이 즉시 프롬프트에 포함된다.
- ③ /config는 클로드 코드의 전역 설정만 변경할 수 있다.
- ④ /doctor는 자동 업데이트 설정 상태를 확인할 수 있다.
- ⑤ /export로 저장한 파일은 CLAUDE.md 작성 시 참고할 수 있다.

2. 다음 상황에서 사용할 적절한 명령어나 기능을 쓰세요.

상황	명령
web_version 폴더 구조를 확인하고 싶을 때	
script.js 파일에서 특정 함수를 찾고 싶을 때	
클로드 코드 설치 상태를 점검하고 싶을 때	
작업한 대화 내용을 파일로 저장하고 싶을 때	
다크 모드로 변경하고 싶을 때	
자동 압축 기능을 끄고 싶을 때	